

Algorithms and Data Structures

Sujeong Kim



Shortest Path

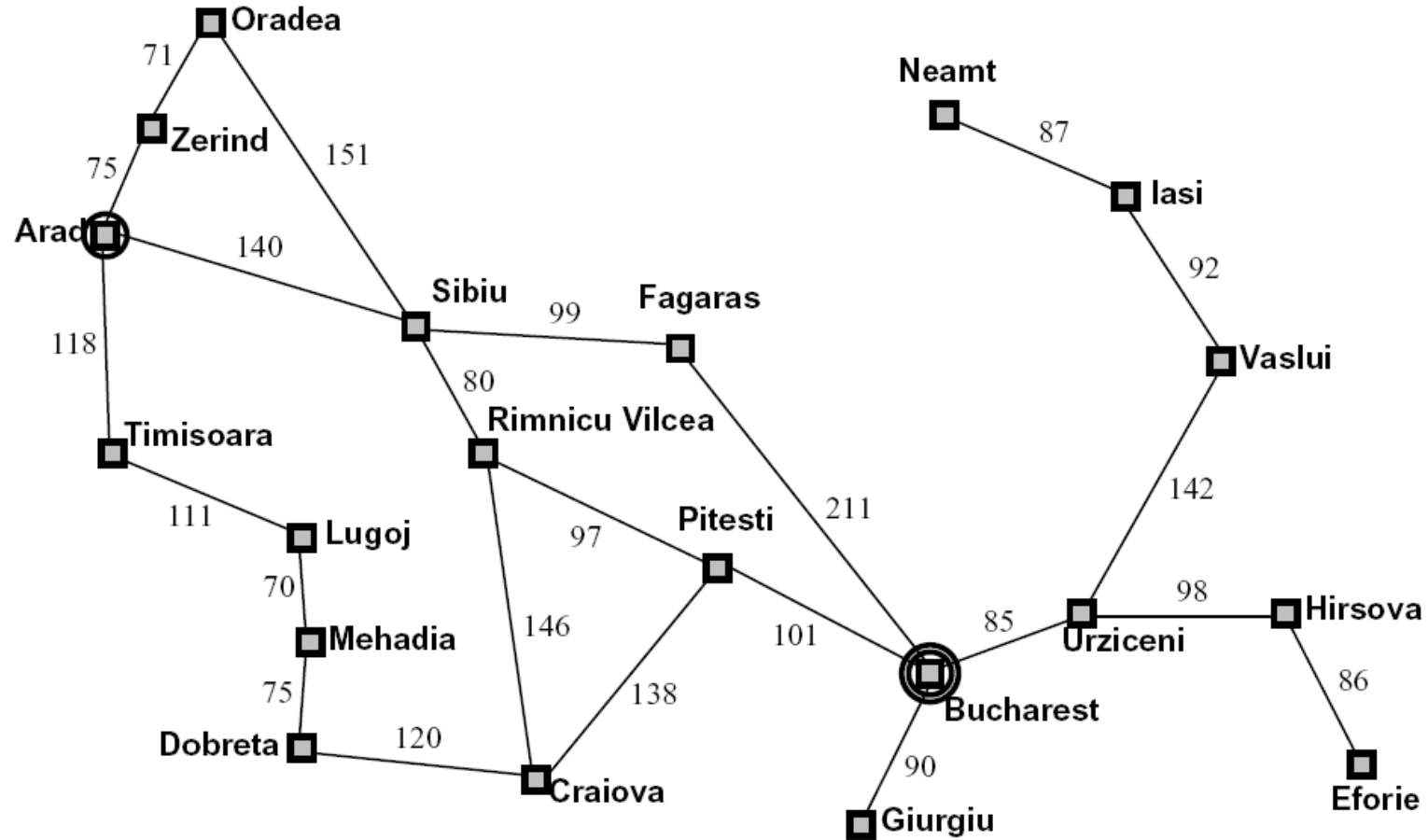
Dijkstra

Slide Credit: These slides are adapted from materials by Prof. Manocha, with modifications and additions by Sujeong Kim.

Definitions for Shortest Paths

- Think of vertices as cities and the edge weights as the distance from one city to another. Define the *length* of a path to be the sum of edge weights along the path. Define the *distance* between two vertices, u and v , $\delta(u, v)$ to be the length of the minimum length path from u to v .
- A *shortest path* from vertex u to vertex v is then defined as any path p with weight $w(p) = \delta(u, v)$.

Search Example: Romania



Single-Source Shortest Paths (SSSP)

- Given a directed graph $G = (V, E)$ with edge weights (which may be positive or negative), compute the minimum-cost distance from a *source vertex*, $s \in V$ to every other vertex in the graph
- Special case: BFS finds short-paths in $O(n+e)$ time, only when all edges have the same weight (typically unweighted graphs)
- The shortest path problem is only well-defined if the graph has no negative-cost cycles
 - If a negative cycle exists, the shortest path is undefined because you can loop forever and reduce the cost

Variants

- *Single-destination shortest-paths problem*: Find a shortest path to a given destination vertex t from every vertex $v \in V$.
- *Single-pair shortest-path problem*: Find a shortest path from u to v for given vertices u and v . If we solve the single-source shortest paths problem, we also solve this problem.
- *All-pairs shortest-paths problem*: Find a shortest path from u to v for every pair of vertices u and v . This can be solved by the single-source problem run for each vertex.

Shortest-paths

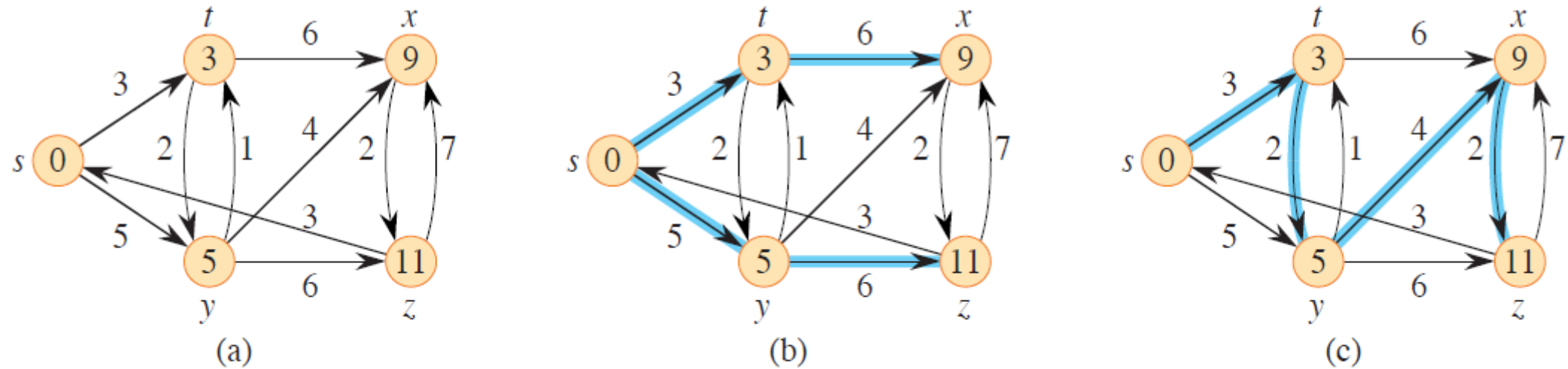
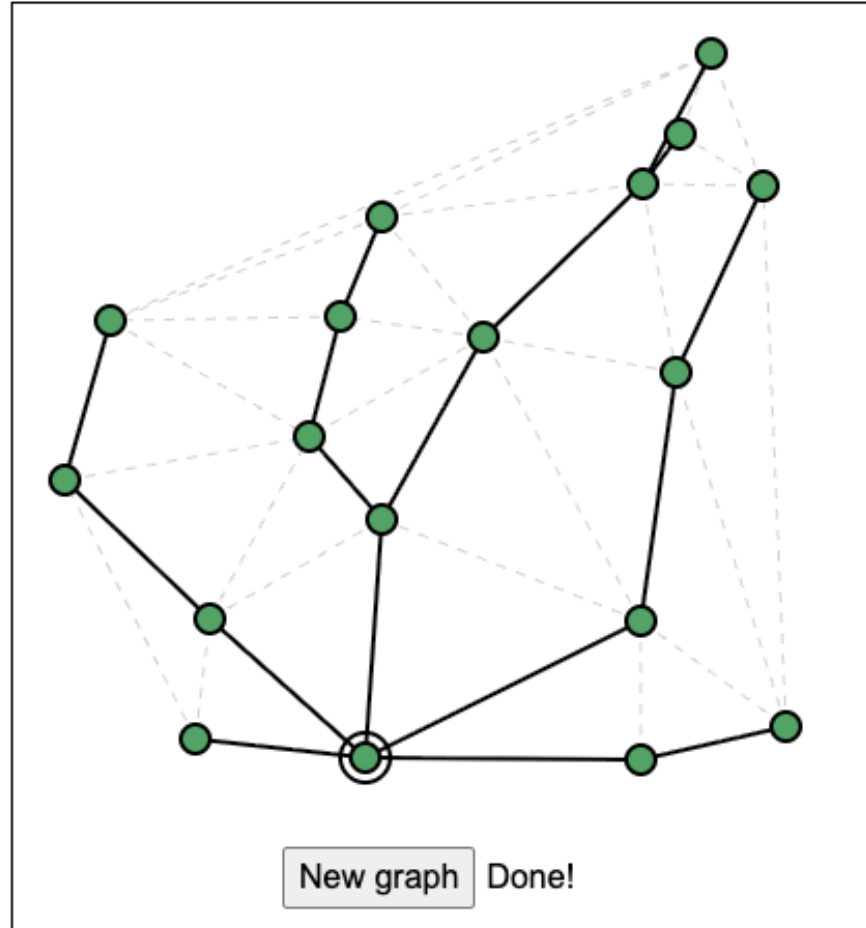


Figure 22.2 (a) A weighted, directed graph with shortest-path weights from source s . (b) The blue edges form a shortest-paths tree rooted at the source s . (c) Another shortest-paths tree with the same root.

Dijkstra's Algorithm demo



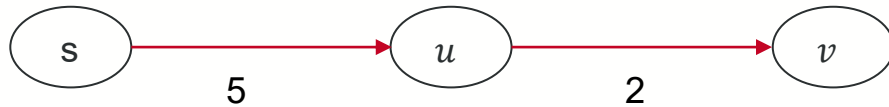
https://www.w3schools.com/dsa/dsa_algo_graphs_dijkstra.php

Relaxation

- Relaxation is the key mechanism that allows algorithms to progressively refine their shortest-path calculation.
- Maintain an estimate of the shortest path for each vertex v , call it $d[v]$.
- $d[v]$ is an upper bound on the weight of a shortest path from source s to v . This value will always be greater than or equal to the true shortest path distance from s to v .
- Initially, we know of no paths, so all $d[v]=\infty$ & $d[s]=0$.
- As the algorithm goes on and sees more vertices, it tries to update $d[v]$ for each vertex in the graph, until all $d[v]$ values converge to true shortest distances.

Find Shortest Path by Relaxation

- This process involves checking if a known path to a vertex can be shortened by taking a different edge. If we find a path from s to v shorter than $d[v]$, then update $d[v]$.
- For each edge (u,v) , if $d[v]$ can be minimized by going through u .
$$d[u] + w(u,v) < d[v]$$



	d	π
s	0	<i>NIL</i>
u	∞	<i>NIL</i>
v	∞	<i>NIL</i>

Initially



	d	π
s	0	<i>NIL</i>
u	5	s
v	∞	<i>NIL</i>



	d	π
s	0	<i>NIL</i>
u	5	s
v	7	u

Relax (u, v, w)

1. if $d[u] + w(u, v) < d[v]$ // is the path thru u shorter?
2. then $d[v] \leftarrow d[u] + w(u, v);$ // yes, then take it.
3. $\pi[v] \leftarrow u;$
// the shortest way back to the source is thru
// u by updating the predecessor pointer

NOTE: If we perform Relax (u, v) repeatedly over all edges of the graph, all the $d[v]$ values will eventually converge to the true final distance values from s . How to do this most efficiently?

Relax (u, v, w) examples

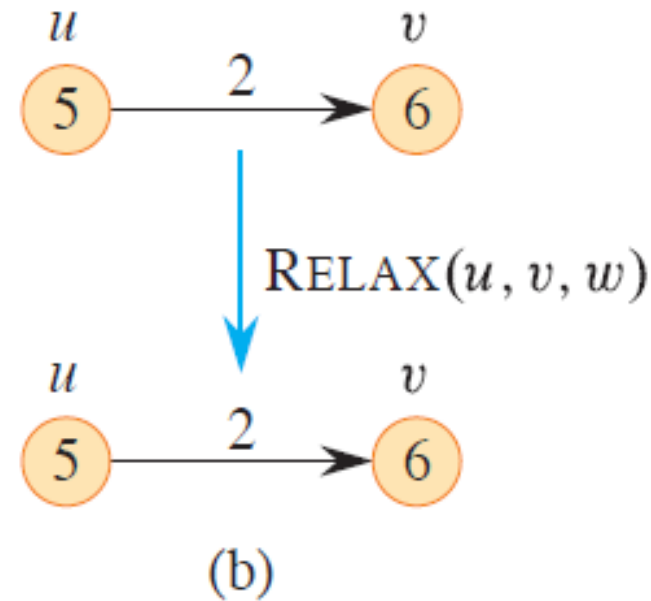
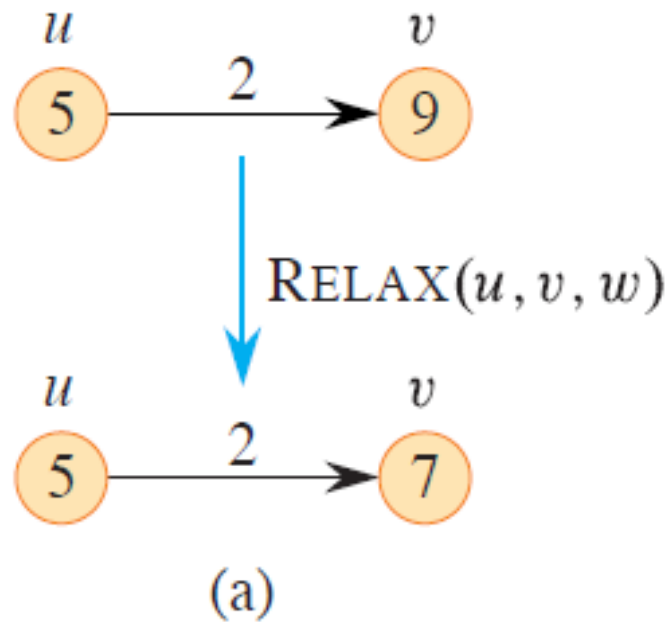


Figure 22.3 Relaxing an edge (u, v) with weight $w(u, v) = 2$. The shortest-path estimate of each vertex appears within the vertex. (a) Because $v.d > u.d + w(u, v)$ prior to relaxation, the value of $v.d$ decreases. (b) Since we have $v.d \leq u.d + w(u, v)$ before relaxing the edge, the relaxation step leaves $v.d$ unchanged.

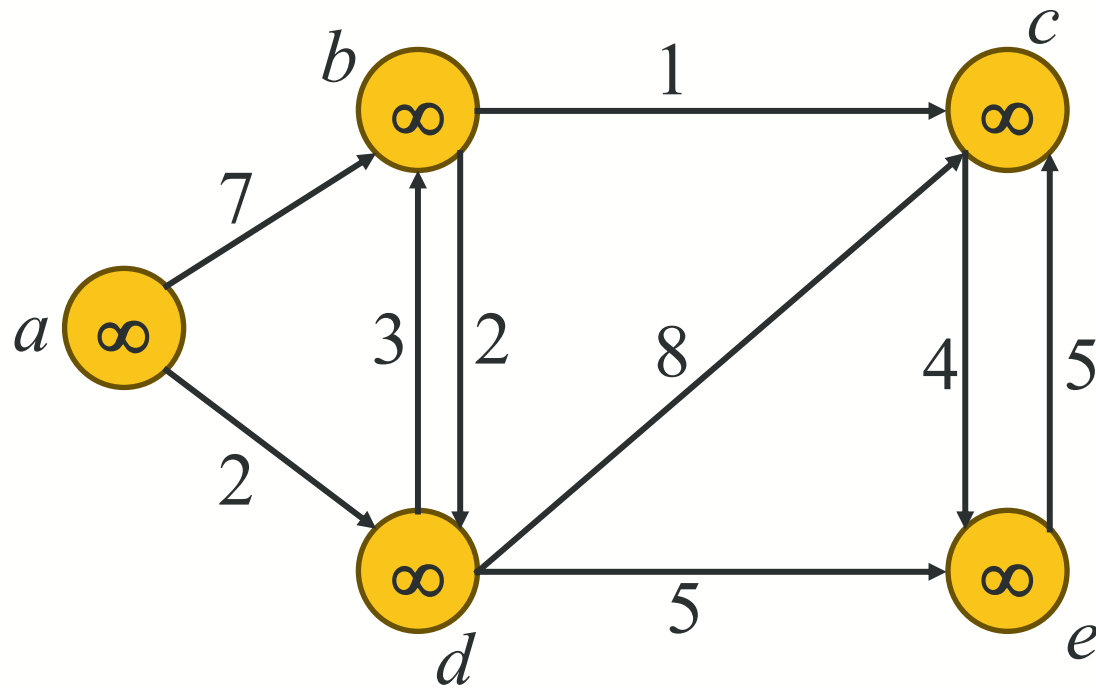
Dijkstra's Algorithm

- Maintains a subset of vertices, $S \subseteq V$, for which we know their true distance $d[u] = \delta(s, u)$. Initially $S = \emptyset$ and we set $d[s] = 0$ and all others to ∞ . One by one we select vertices from $V - S$ to add to S .
- For each vertex $u \in V - S$, we have computed a distance estimate $d[u]$. The greedy approach is to take the vertex for which $d[u]$ is minimum, i.e. take unprocessed vertex that is closest to s .
- We store the vertices of $V - S$ in a priority queue (heap), where the key value of each vertex u is $d[u]$. All operations can be done in $O(\lg n)$ time.

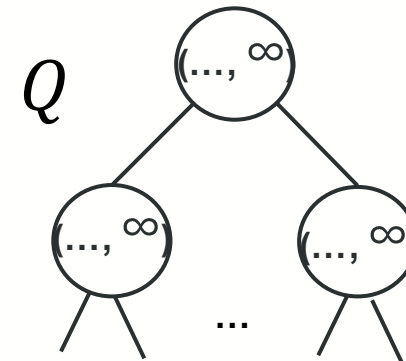
Dijkstra(G, w, s)

```
1.  $Q \leftarrow V[G]$  and  $S \leftarrow \emptyset$ 
2. for each vertex  $u \in Q$            // initialization:  $O(V)$  time
3.     do  $d[u] \leftarrow \infty$  and  $\pi[u] \leftarrow \text{NIL}$ 
4.  $d[s] \leftarrow 0$                  // start at the source
5.  $\pi[s] \leftarrow \text{NIL}$            // set parent of  $s$  to be NIL
6. while  $Q \neq \emptyset$            // till all vertices processed
7.     do  $u \leftarrow \text{Extract-Min}(Q)$  // select closest to  $s$ 
         $S \leftarrow S \cup \{u\}$ 
8.     for each  $v \in \text{adj}[u]$ 
9.         do if  $v \in Q$  and  $(d[u] + w(u,v) < d[v])$ 
10.            then  $\pi[v] \leftarrow u$ 
11.                 $d[v] \leftarrow d[u] + w(u,v)$  // Relax  $(u,v)$ 
12.                decrease_Key( $Q, v, d[v]$ )
```

Example: Dijkstra's Algorithm



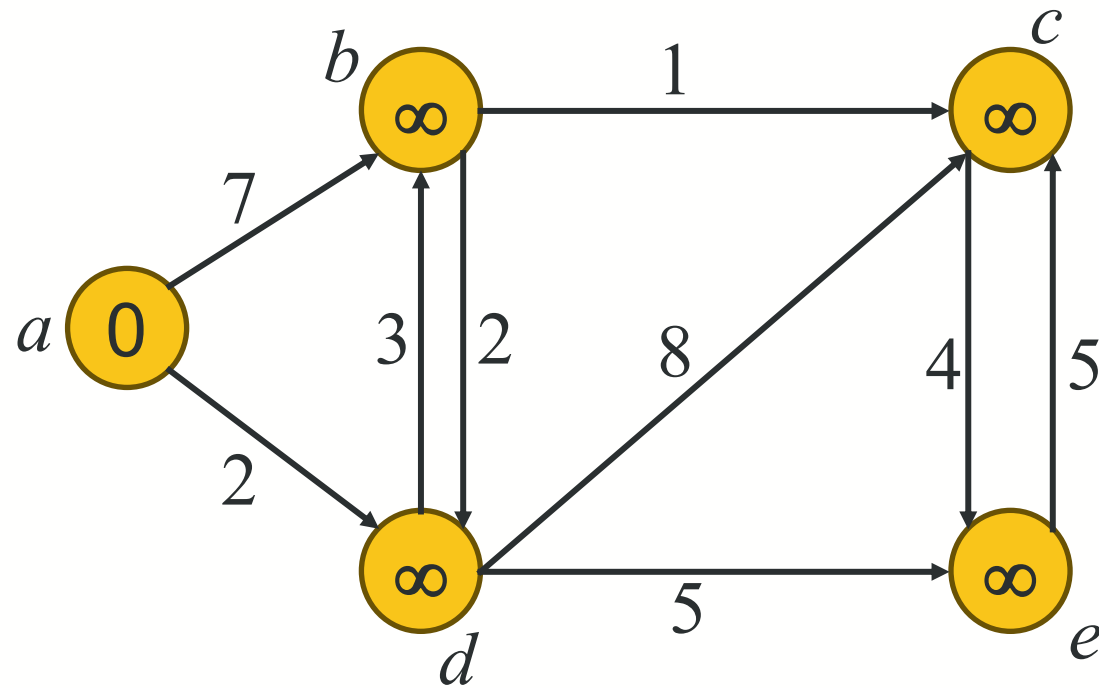
1. $Q \leftarrow V[G]$ and $S \leftarrow \emptyset$
2. for each vertex $u \in Q$
3. do $d[u] \leftarrow \infty$ and $\pi[u] \leftarrow \text{NIL}$



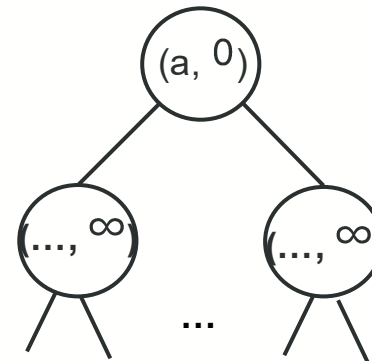
$S = \{\}$

	d	π
a	∞	NIL
b	∞	NIL
c	∞	NIL
d	∞	NIL
e	∞	NIL

Example: Dijkstra's Algorithm



4. $d[s] \leftarrow 0$
5. $\pi[s] \leftarrow \text{NIL}$



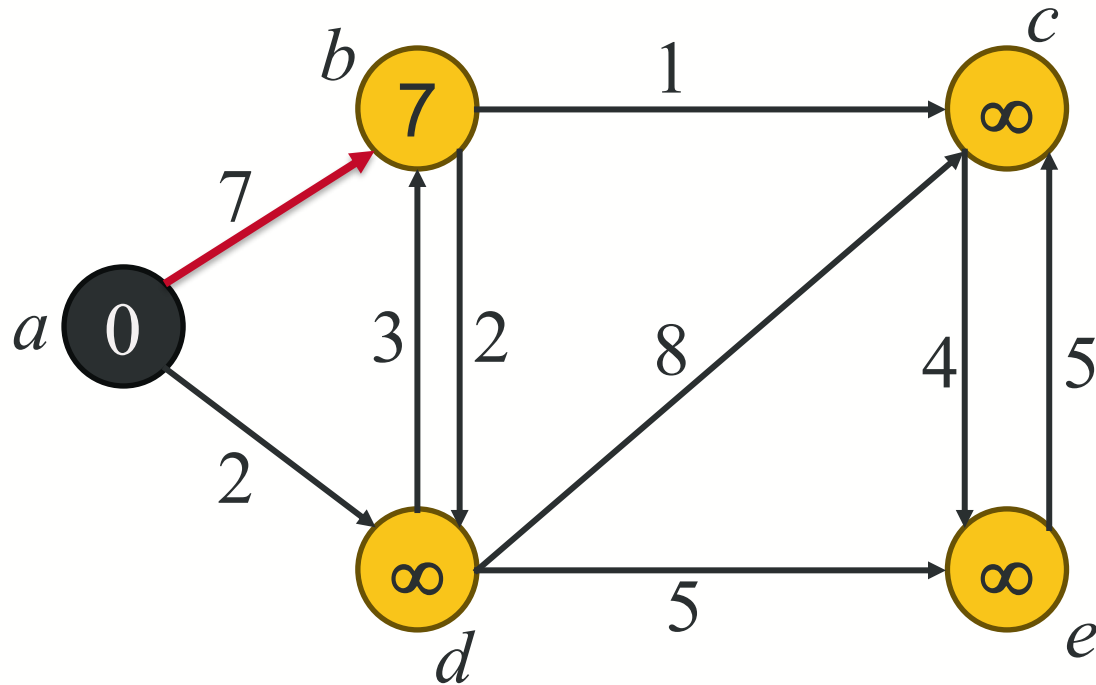
$S = \{ \}$

	d	π
a	0	NIL
b	∞	NIL
c	∞	NIL
d	∞	NIL
e	∞	NIL

Example: Dijkstra's Algorithm

$d[b] = \infty$

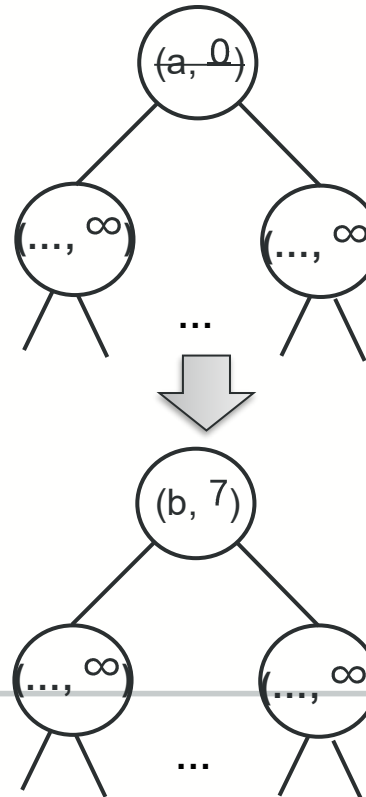
$d[a] + w(a,b) = 7$ // $a \rightarrow b$



6. while $Q \neq \emptyset$
7. do $u \leftarrow \text{Extract-Min}(Q)$
 $S \leftarrow S \cup \{u\}$
8. for each $v \in \text{adj}[u]$
9. do if $v \in Q$ and $(d[u] + w(u,v) < d[v])$
10. then $\pi[v] \leftarrow u$
11. $d[v] \leftarrow d[u] + w(u,v)$ // Relax (u,v)
12. decrease_Key($Q, v, d[v]$)

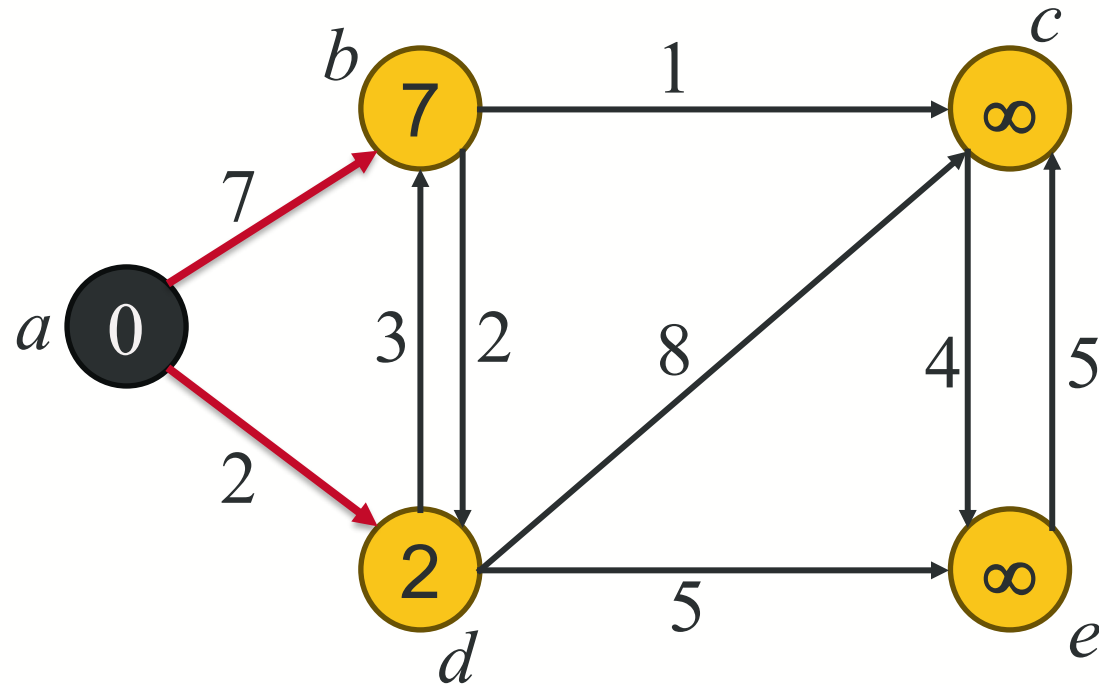
$S = \{a\}$
 $\text{adj}[a] = \{b, d\}$
 $v = b$

	d	π
a	0	NIL
b	7	a
c	∞	NIL
d	∞	NIL
e	∞	NIL



Black: in S

Example: Dijkstra's Algorithm

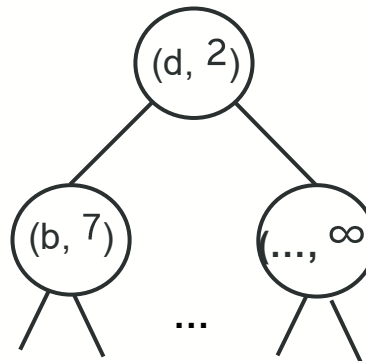


$d[d] = \infty$

$d[a] + w(a,d) = 2$ // $a \rightarrow d$

6. while $Q \neq \emptyset$
7. do $u \leftarrow \text{Extract-Min}(Q)$
 $S \leftarrow S \cup \{u\}$
8. for each $v \in \text{adj}[u]$
9. do if $v \in Q$ and $(d[u] + w(u,v) < d[v])$
10. then $\pi[v] \leftarrow u$
11. $d[v] \leftarrow d[u] + w(u,v)$ // Relax (u,v)
12. decrease_Key($Q, v, d[v]$)

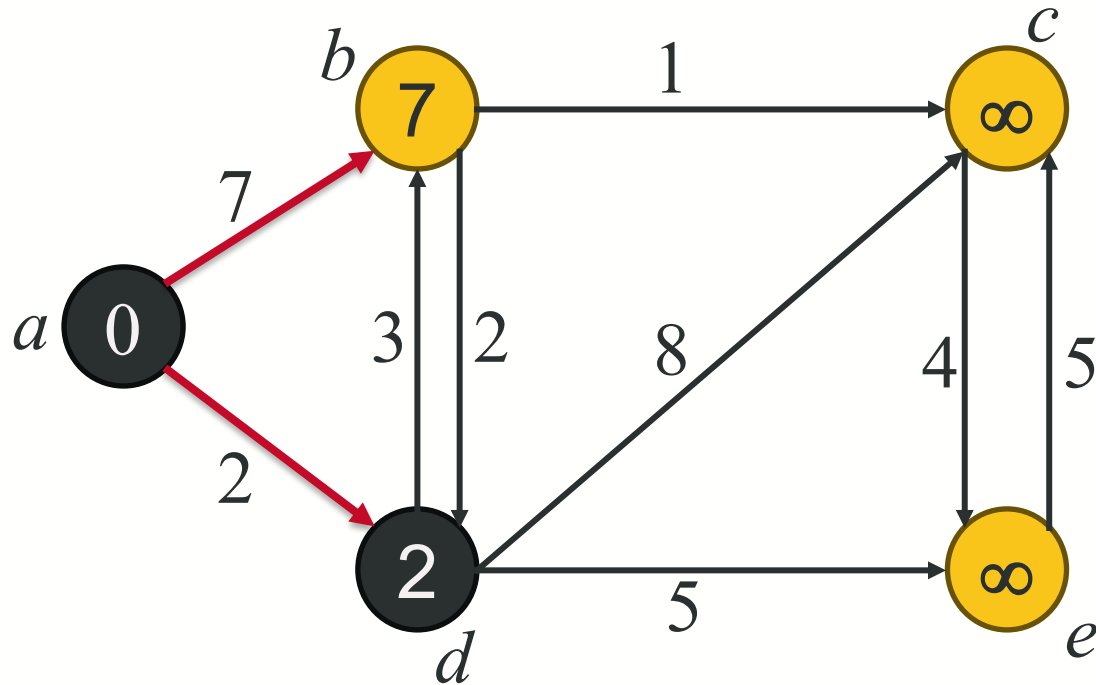
$S = \{a\}$
 $\text{adj}[a] = \{b, d\}$
 $v = d$



	d	π
a	0	NIL
b	7	a
c	∞	NIL
d	2	a
e	∞	NIL

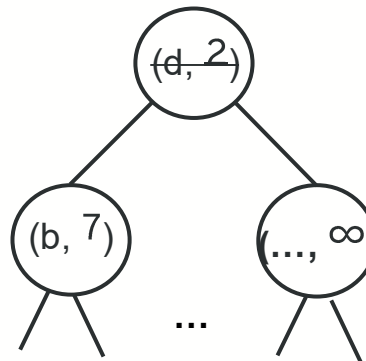
Black: in S

Example: Dijkstra's Algorithm



6. while $Q \neq \emptyset$
7. do $u \leftarrow \text{Extract-Min}(Q)$
 $S \leftarrow S \cup \{u\}$
8. for each $v \in \text{adj}[u]$
9. do if $v \in Q$ and $(d[u] + w(u,v) < d[v])$
10. then $\pi[v] \leftarrow u$
11. $d[v] \leftarrow d[u] + w(u,v)$ // Relax (u,v)
12. decrease_Key($Q, v, d[v]$)

$S = \{a, d\}$
 $\text{adj}[d] = \{b, c, e\}$

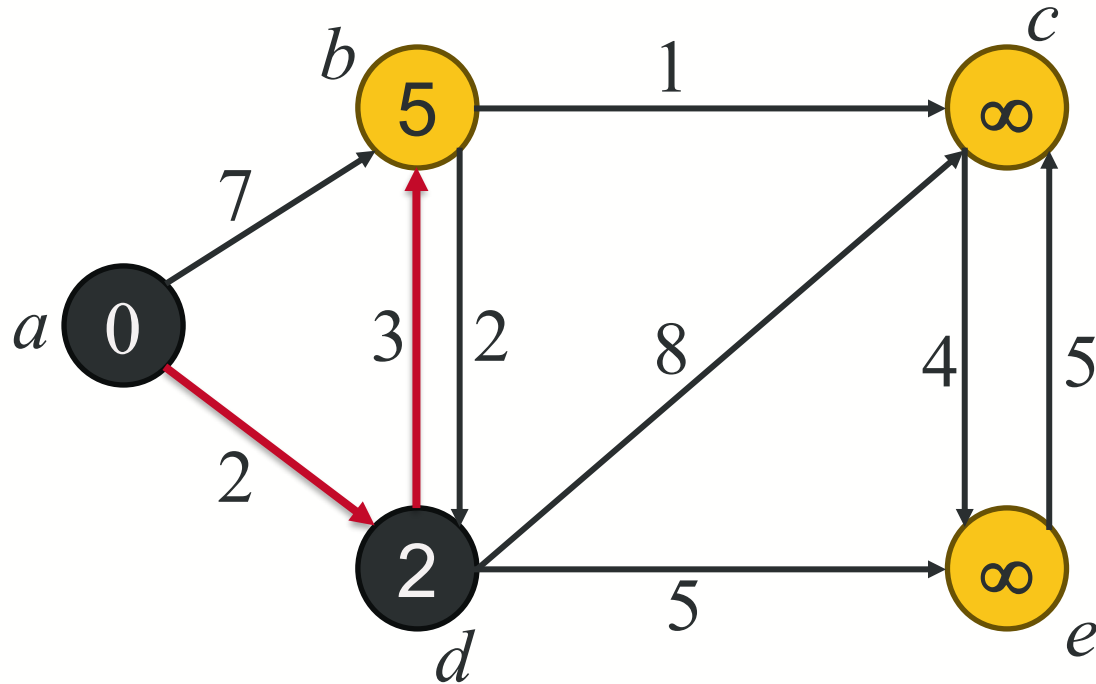


Black: in S

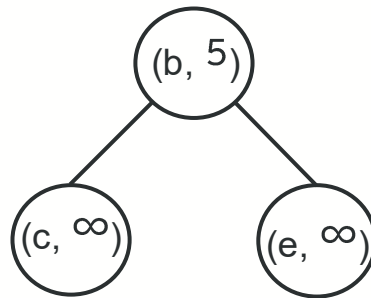
Example: Dijkstra's Algorithm

$d[b]=7$ // $a \rightarrow b$

$d[d]+w(d,b) = 2+3 = 5$ // $a \rightarrow d \rightarrow b$



6. while $Q \neq \emptyset$
7. do $u \leftarrow \text{Extract-Min}(Q)$
 $S \leftarrow S \cup \{u\}$
8. for each $v \in \text{adj}[u]$
9. do if $v \in Q$ and $(d[u] + w(u,v) < d[v])$
10. then $\pi[v] \leftarrow u$
11. $d[v] \leftarrow d[u] + w(u,v)$ // Relax (u,v)
12. decrease_Key($Q, v, d[v]$)



$S = \{a, d\}$

$\text{adj}[d] = \{b, c, e\}$

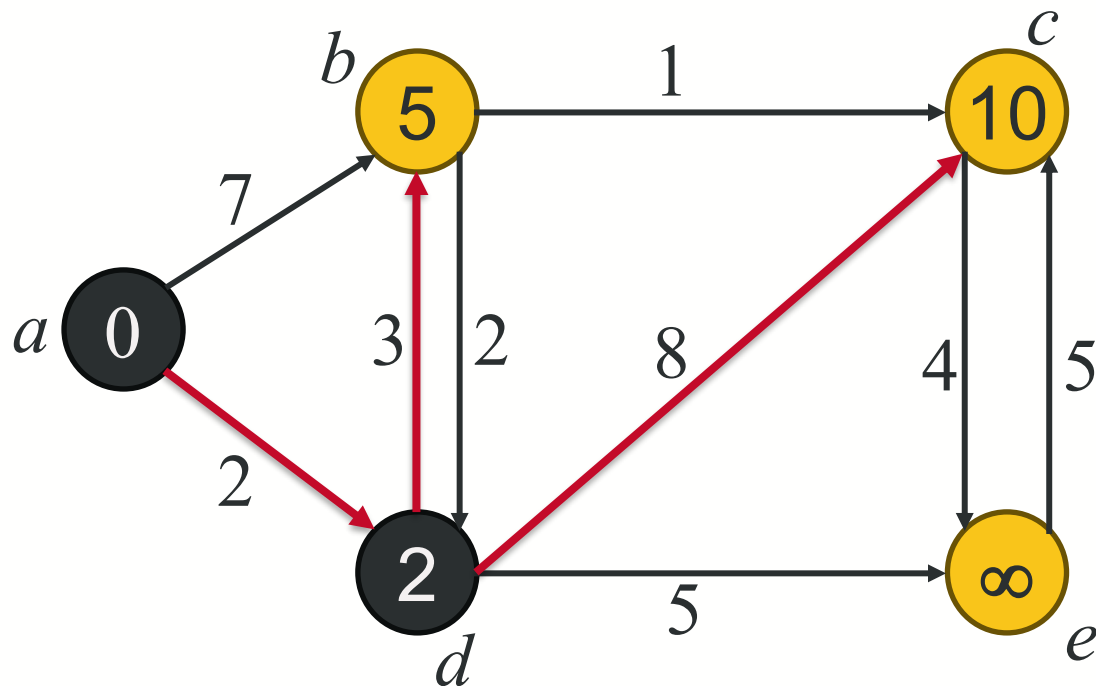
$v = b$

	d	π
a	0	NIL
b	5	d
c	∞	NIL
d	2	a
e	∞	NIL

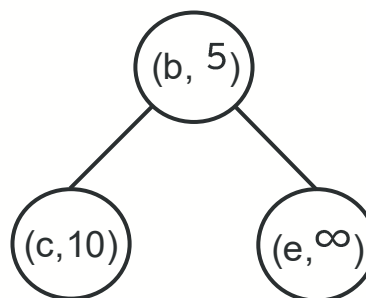
Black: in S

Example: Dijkstra's Algorithm

$d[c] = \infty$
 $d[d] + w(d, c) = 2 + 8 = 10$ // $a \rightarrow d \rightarrow c$



6. while $Q \neq \emptyset$
7. do $u \leftarrow \text{Extract-Min}(Q)$
 $S \leftarrow S \cup \{u\}$
8. for each $v \in \text{adj}[u]$
9. do if $v \in Q$ and $(d[u] + w(u, v) < d[v])$
10. then $\pi[v] \leftarrow u$
11. $d[v] \leftarrow d[u] + w(u, v)$ // Relax (u, v)
12. decrease_Key($Q, v, d[v]$)

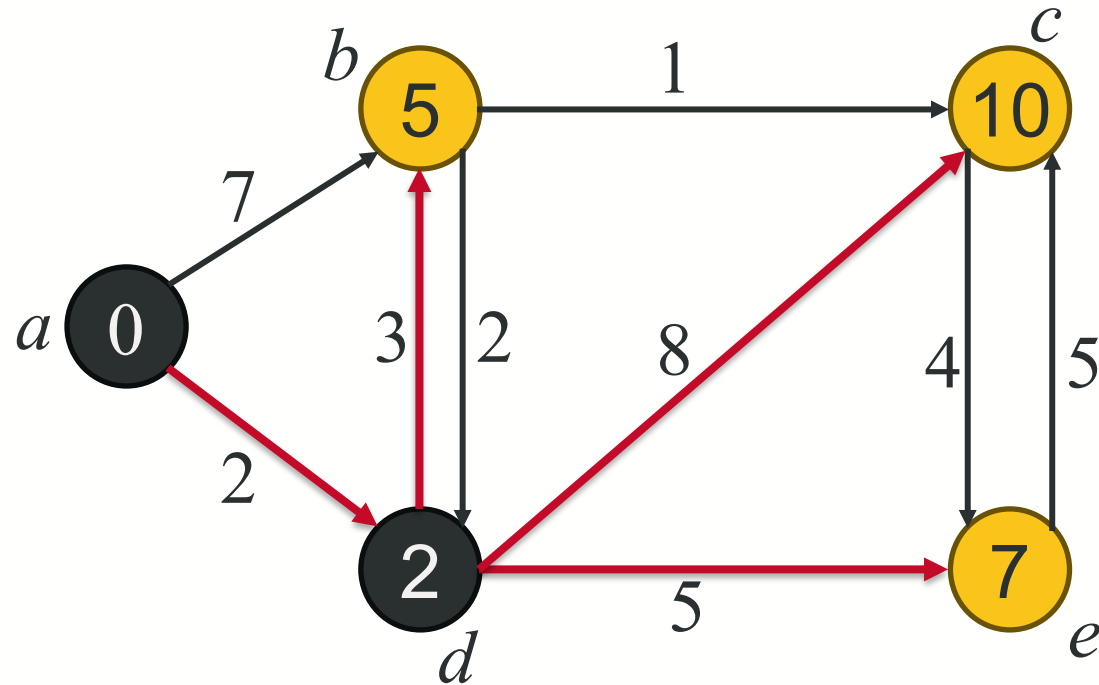


$S = \{a, d\}$
 $\text{adj}[d] = \{b, c, e\}$
 $v = c$

	d	π
a	0	NIL
b	5	d
c	10	d
d	2	a
e	∞	NIL

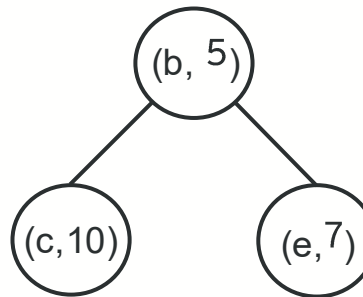
Black: in S

Example: Dijkstra's Algorithm



$d[e] = \infty$
 $d[d] + w(d, e) = 2 + 5 = 7$ // $a \rightarrow d \rightarrow e$

6. while $Q \neq \emptyset$
7. do $u \leftarrow \text{Extract-Min}(Q)$
 $S \leftarrow S \cup \{u\}$
8. for each $v \in \text{adj}[u]$
9. do if $v \in Q$ and $(d[u] + w(u, v) < d[v])$
10. then $\pi[v] \leftarrow u$
11. $d[v] \leftarrow d[u] + w(u, v)$ // Relax (u, v)
12. decrease_Key($Q, v, d[v]$)

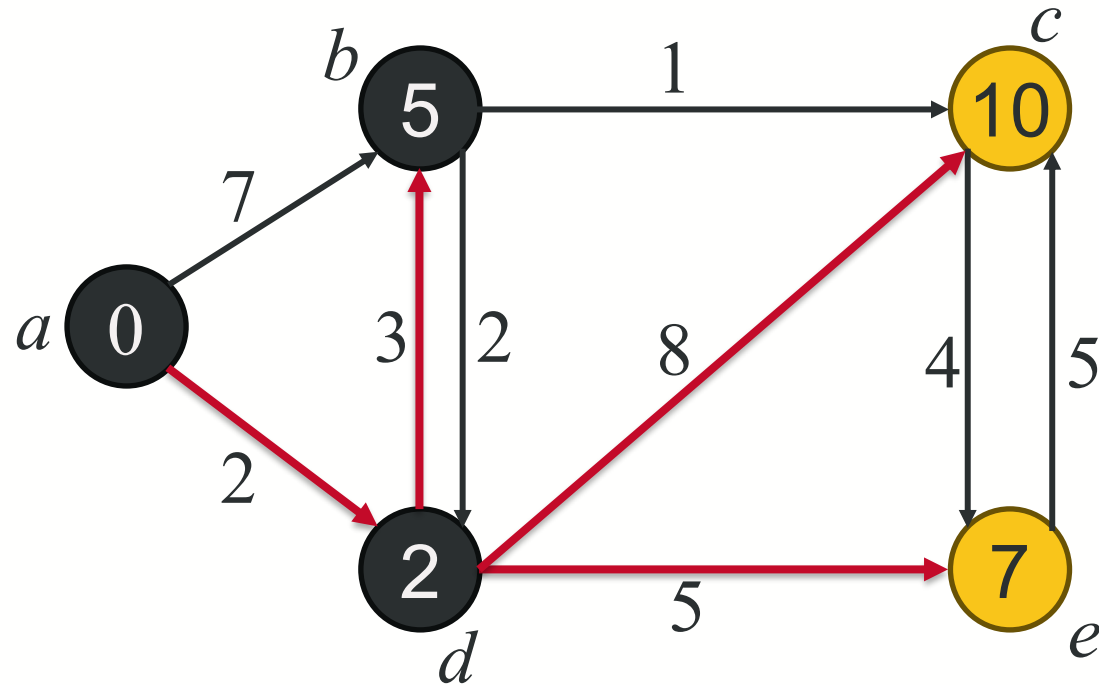


$S = \{a, d\}$
 $\text{adj}[d] = \{b, c, e\}$
 $v = e$

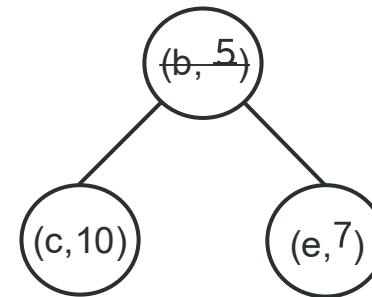
	d	π
a	0	NIL
b	5	d
c	10	d
d	2	a
e	7	d

Black: in S

Example: Dijkstra's Algorithm



6. while $Q \neq \emptyset$
7. do $u \leftarrow \text{Extract-Min}(Q)$
 $S \leftarrow S \cup \{u\}$
8. for each $v \in \text{adj}[u]$
9. do if $v \in Q$ and $(d[u] + w(u,v) < d[v])$
10. then $\pi[v] \leftarrow u$
11. $d[v] \leftarrow d[u] + w(u,v)$ // Relax (u,v)
12. decrease_Key($Q, v, d[v]$)



$S = \{a, d, b\}$
 $\text{adj}[b] = \{c, d\}$

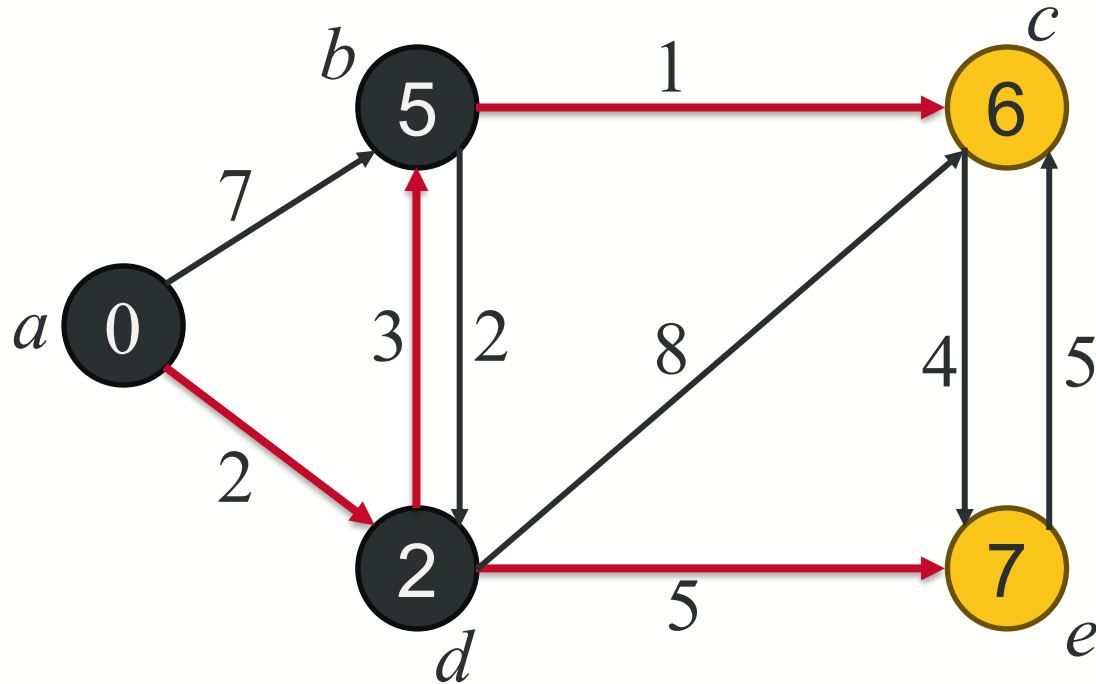
	d	π
a	0	NIL
b	5	d
c	10	d
d	2	a
e	7	d

Black: in S

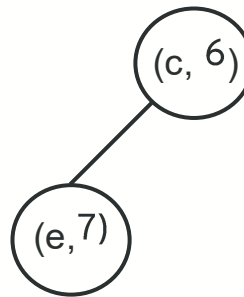
Example: Dijkstra's Algorithm

$d[c]=10$ // $a \rightarrow d \rightarrow c$

$d[b]+w(b,c) = 5+1=6$ // $a \rightarrow d \rightarrow b \rightarrow c$



6. while $Q \neq \emptyset$
7. do $u \leftarrow \text{Extract-Min}(Q)$
 $S \leftarrow S \cup \{u\}$
8. for each $v \in \text{adj}[u]$
9. do if $v \in Q$ and $(d[u] + w(u,v) < d[v])$
10. then $\pi[v] \leftarrow u$
11. $d[v] \leftarrow d[u] + w(u,v)$ // Relax (u,v)
12. decrease_Key($Q, v, d[v]$)

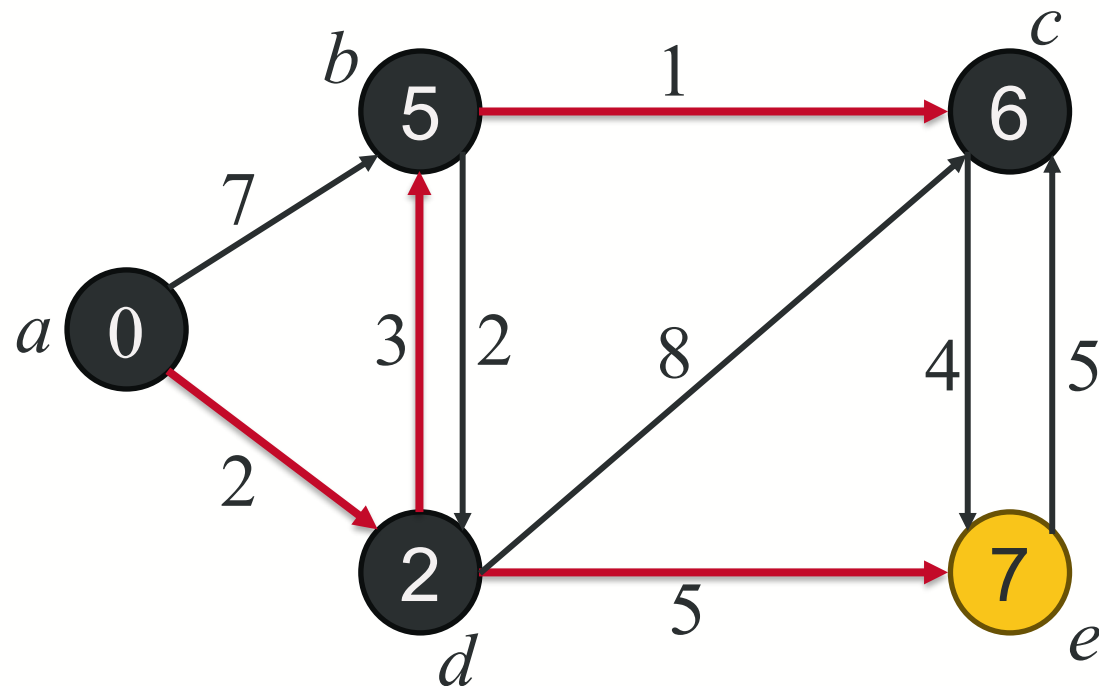


$S = \{a, d, b\}$
 $\text{adj}[b] = \{c, d\}$
 $u = b$
 $v = c$

	d	π
a	0	NIL
b	5	d
c	6	b
d	2	a
e	7	d

Black: in S

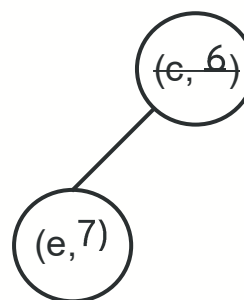
Example: Dijkstra's Algorithm



6. while $Q \neq \emptyset$
7. do $u \leftarrow \text{Extract-Min}(Q)$
 $S \leftarrow S \cup \{u\}$
8. for each $v \in \text{adj}[u]$
9. do if $v \in Q$ and $(d[u] + w(u,v) < d[v])$
10. then $\pi[v] \leftarrow u$
11. $d[v] \leftarrow d[u] + w(u,v)$ // Relax (u,v)
12. decrease_Key($Q, v, d[v]$)

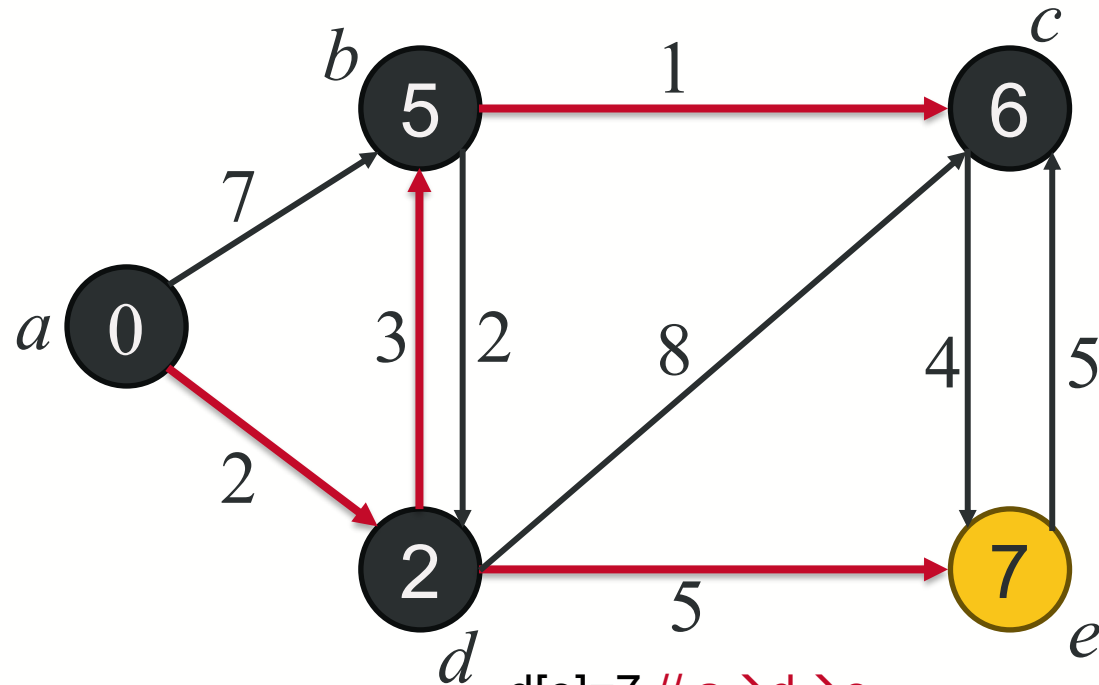
$S = \{a, d, b, c\}$
 $\text{adj}[c] = \{e\}$

	d	π
a	0	NIL
b	5	d
c	6	b
d	2	a
e	7	d



Black: in S

Example: Dijkstra's Algorithm



$d[e]=7$ // $a \rightarrow d \rightarrow e$

$d[c]+w(c,e) = 6+4=10$ // $a \rightarrow d \rightarrow b \rightarrow c \rightarrow e$

6. while $Q \neq \emptyset$
7. do $u \leftarrow \text{Extract-Min}(Q)$
 $S \leftarrow S \cup \{u\}$
8. for each $v \in \text{adj}[u]$
9. do if $v \in Q$ and $(d[u] + w(u,v) < d[v])$
10. then $\pi[v] \leftarrow u$
11. $d[v] \leftarrow d[u] + w(u,v)$ // Relax (u,v)
12. decrease_Key($Q, v, d[v]$)

$S = \{a, d, b, c\}$
 $\text{adj}[c] = \{e\}$

$u = c$

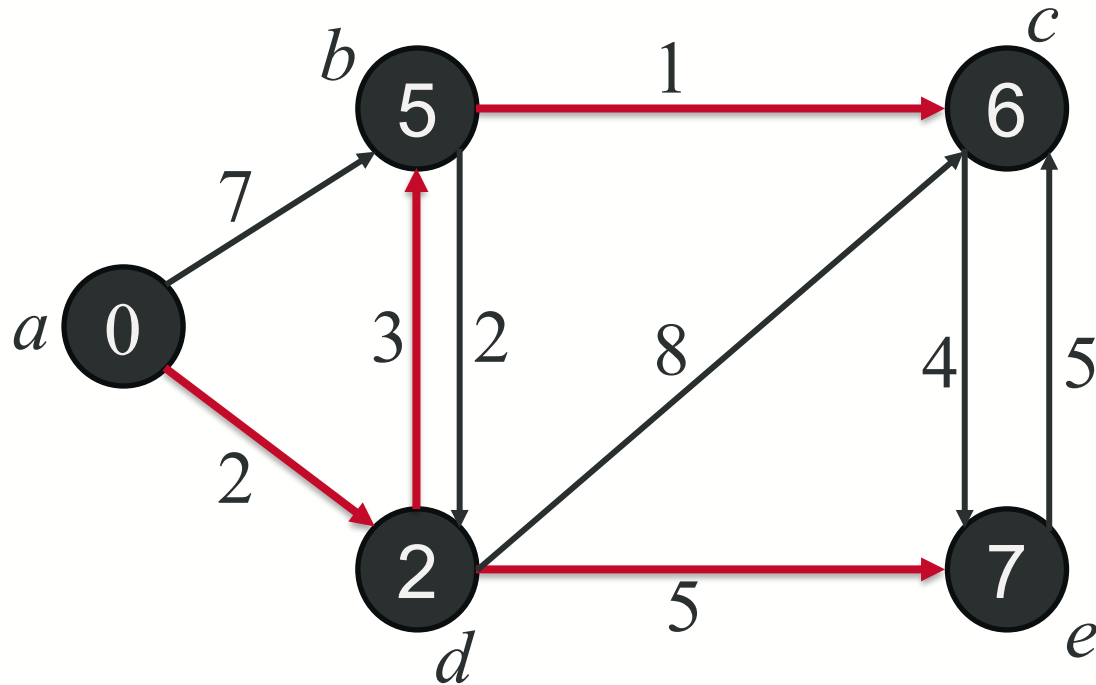
$v = e$

(e,7)

	d	π
a	0	NIL
b	5	d
c	6	b
d	2	a
e	7	d

Black: in S

Example: Dijkstra's Algorithm



6. while $Q \neq \emptyset$
7. do $u \leftarrow \text{Extract-Min}(Q)$
 $S \leftarrow S \cup \{u\}$
8. for each $v \in \text{adj}[u]$
9. do if $v \in Q$ and $(d[u] + w(u,v) < d[v])$
10. then $\pi[v] \leftarrow u$
11. $d[v] \leftarrow d[u] + w(u,v)$ // Relax (u,v)
12. decrease_Key($Q, v, d[v]$)

$S = \{a, d, b, c\}$
 $\text{adj}[e] = \{c\}$

	d	π
a	0	NIL
b	5	d
c	6	b
d	2	a
e	7	d

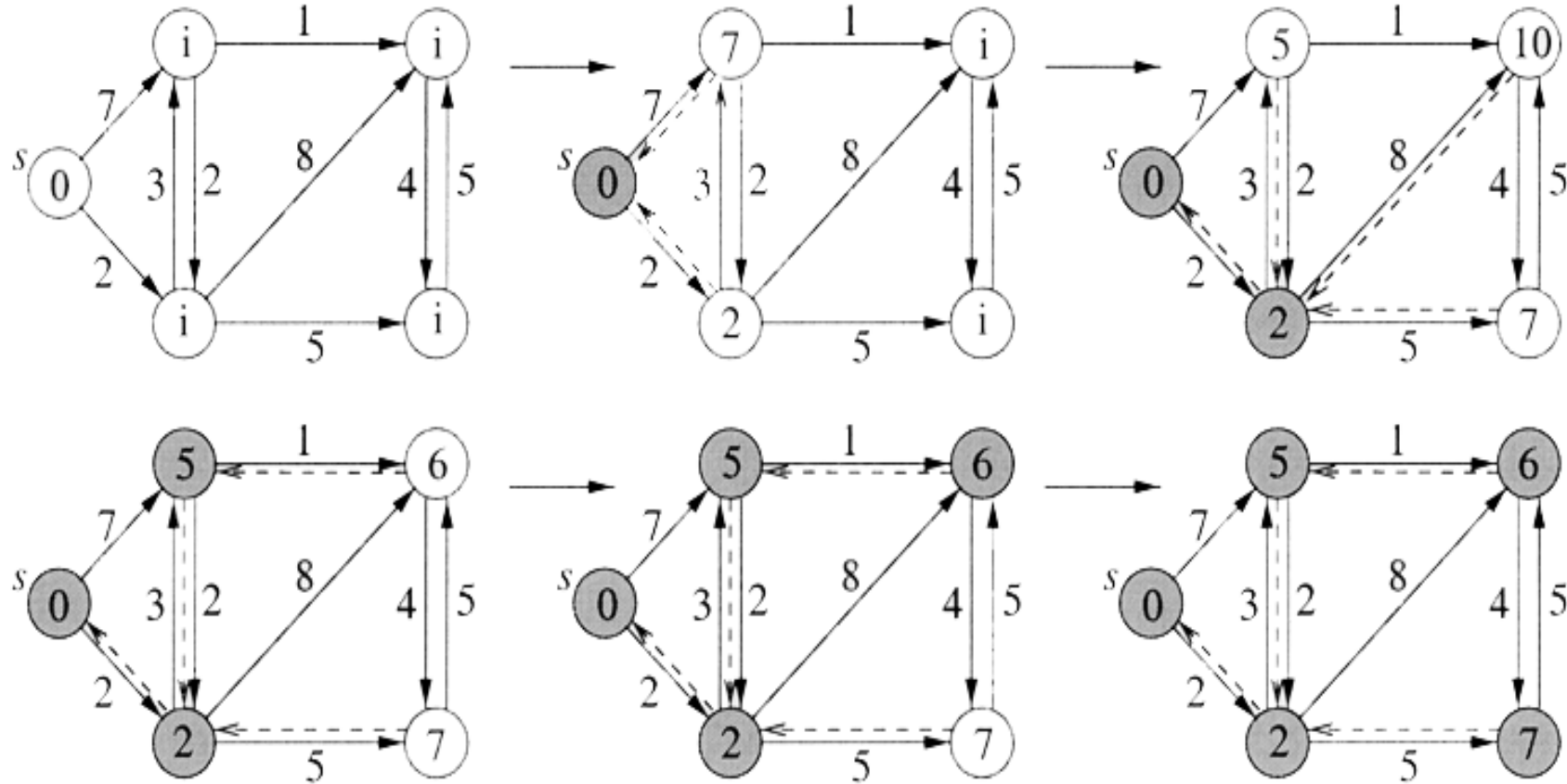
(e,7)



$Q \neq \emptyset$

Black: in S

The Sequence of Relaxations



(Think-pair-share) BFS vs. Dijkstra

1. What does “shortest path” mean? What quantity does each algorithm actually minimize?
2. When is BFS valid for shortest path, and when is Dijkstra required instead?
3. When is the shortest path to a node finalized?
4. Why does BFS never need to revisit or update a node after discovery (i.e., relax), while Dijkstra might?
5. What data structure does each use, and how does that affect runtime?
6. Special cases:
 - What happens if all edges weights are equal (but not necessarily 1)?
 - What happens if all edge weights are 0?
 - What happens if the graph contains a negative edge?
7. (True/False) Both algorithms compute shortest paths from the start node to all reachable nodes
8. (True/False) Both algorithms work on both directed and undirected graphs

Correctness

- Let $\delta(s,v)$ be length of true shortest path from s to v . Need to show: $d[v] = \delta(s,v)$ for each v when the algorithm terminates
- Key properties
 - Choosing the closest unprocessed node guarantees there isn't a shorter path to it at that moment.
 - If a route is the shortest way to a destination, then each part of that route is also the shortest possible for its selection. This allows Dijkstra's Algorithm to solve the problem step by step.

Correctness

- Let $\delta(s,v)$ be length of true shortest path from s to v . Need to show:
 $d[v] = \delta(s,v)$ for each v when the algorithm terminates

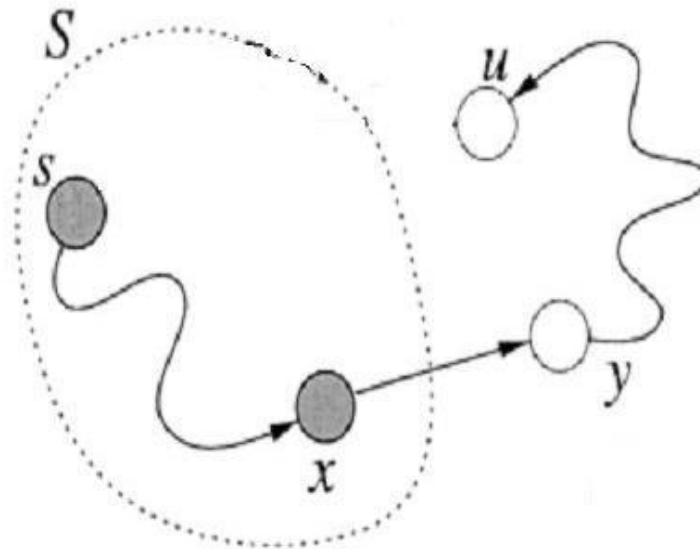
Lemma: When a vertex u is added to S , $d[u] = \delta(s,u)$.

Proof:

- Assume Dijkstra is about to add a vertex u to S but $d[u] \neq \delta(s,u)$
 - Note $d[u]$ is never less than $\delta(s,u)$, thus $d[u] > \delta(s,u)$
- Show that this assumption leads to a contradiction

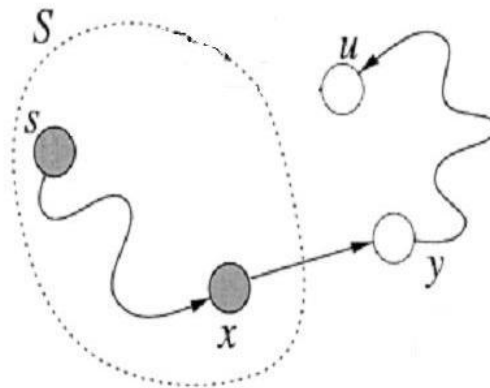
Proof (I)

- Just prior to the insertion of u , consider the true shortest path from s to u .
- Because $s \in S$ and $u \in V - S$, at some point this path must first jump out of S .
- Let (x, y) be the edge where it jumps out, so that $x \in S$ and $y \in V - S$. (It might happen that $x=s$ and/or $y=u$).



Proof (II): $y \neq u$

- We argue that $y \neq u$ after all
 - Since $x \in S$ we have $d[x] = \delta(s, x)$. (Remember that u was the first vertex added to S that violated this criterion.)
 - Since we applied relaxation to x when it was added, we would have set $d[y] = d[x] + w(x, y) = \delta(s, y)$.
 - Since (x, y) is on a shortest path from s to u , it is on a shortest path from s to y . Thus $d[y]$ is now correct.
 - By hypothesis, $d[u]$ is not correct, so u and y cannot be the same.



Conclusion of Proof

- Notice:
 - y appears somewhere along the shortest path from s to u (but not at u)
 - All subsequent edges following y are of weight ≥ 0 ,
- Therefore, $\delta(s,y) \leq \delta(s,u)$, so $d[y] = \delta(s,y) \leq \delta(s,u) < d[u]$
- But, $d[u] \leq d[y]$ because u is added before y , and we add nodes with lower d values first.

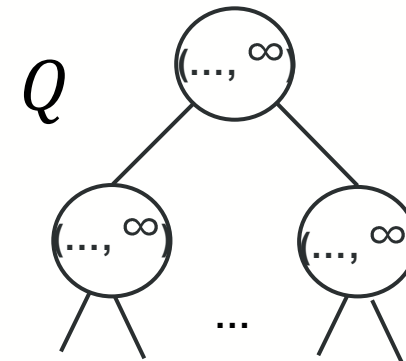
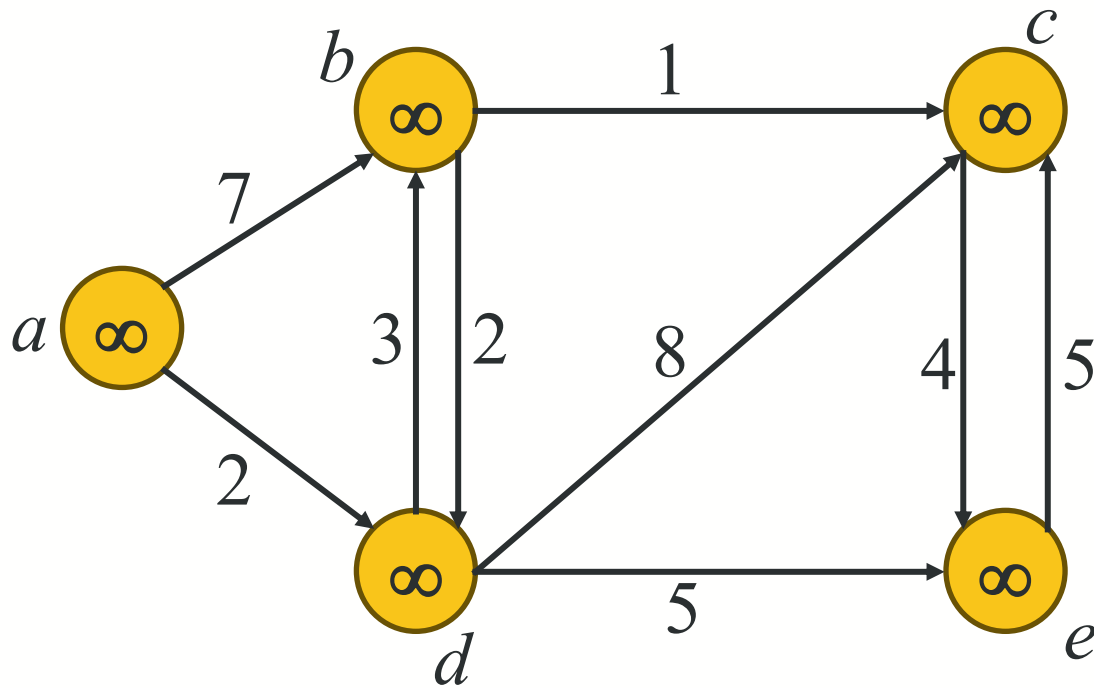
Contradiction!

Correctness

- We have just proved that, when a vertex u is added to S , $d[u] = \delta(s, u)$.
- Every vertex is eventually added to S , so the algorithm assigns the correct distances to all vertices.
- This guarantees that the shortest-paths tree is correct because relaxation correctly updates parent pointers (see book for details).

How is the heap (Q) used in Dijkstra's algorithm?

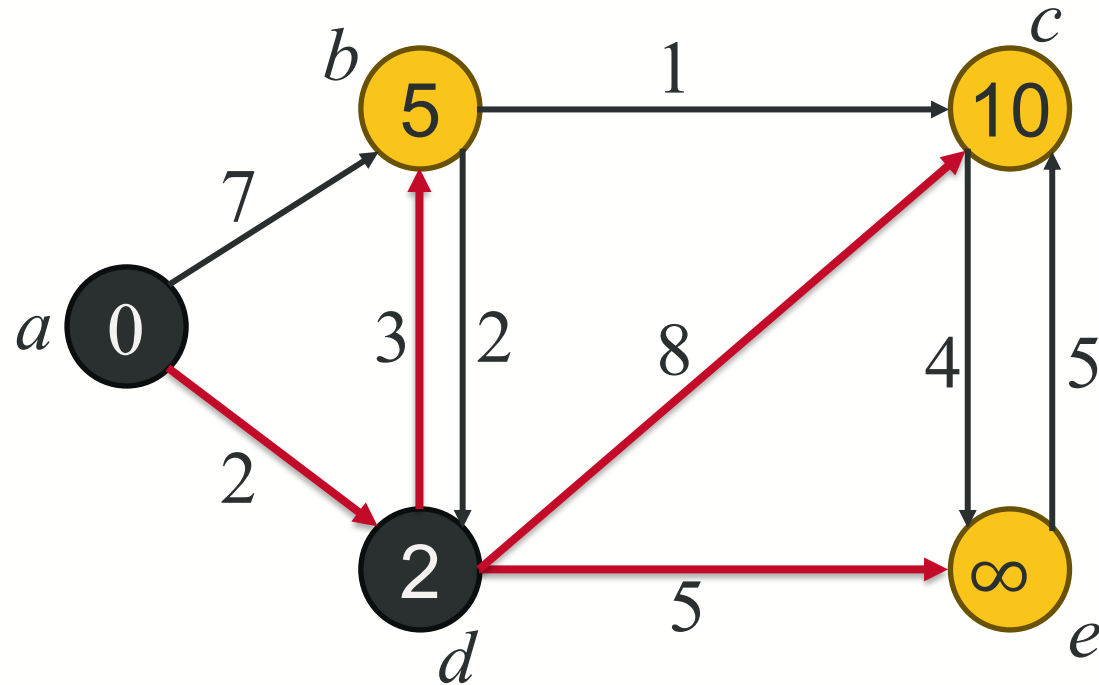
1. $Q \leftarrow V[G]$ and $S \leftarrow \emptyset$
2. for each vertex $u \in Q$
3. do $d[u] \leftarrow \infty$ and $\pi[u] \leftarrow \text{NIL}$



$S = \{\}$

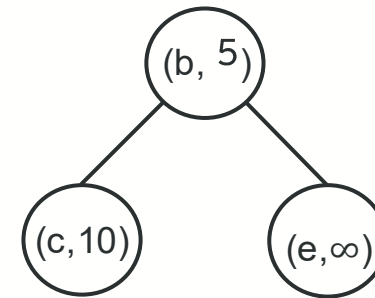
	d	π
a	∞	NIL
b	∞	NIL
c	∞	NIL
d	∞	NIL
e	∞	NIL

Extract-Min(Q)

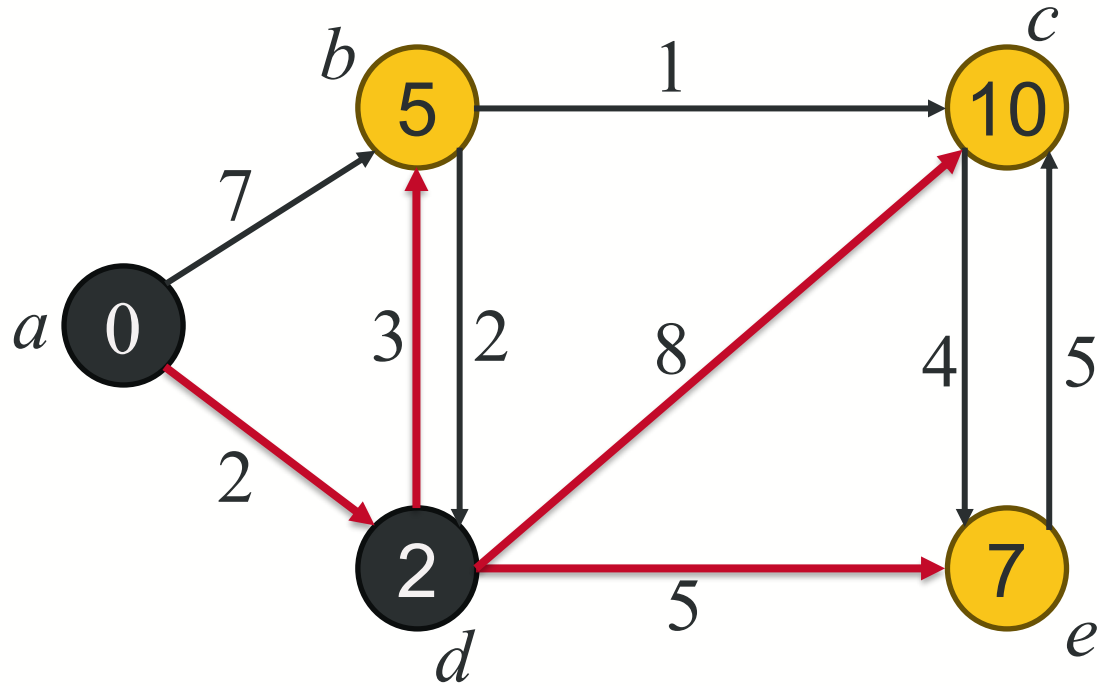


$d[e] = \infty$
 $d[d] + w(d,e) = 2 + 5 = 7$ // $a \rightarrow d \rightarrow e$

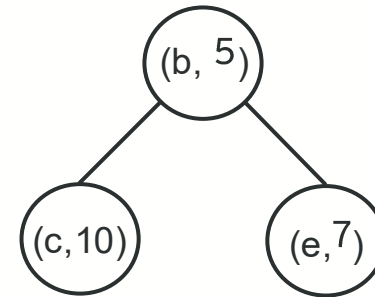
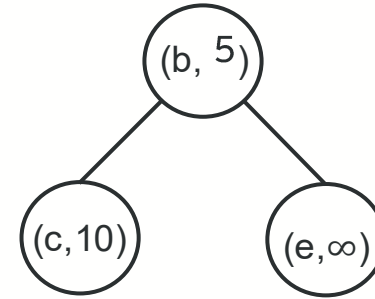
6. while $Q \neq \emptyset$
7. do $u \leftarrow \text{Extract-Min}(Q)$
 $S \leftarrow S \cup \{u\}$
8. for each $v \in \text{adj}[u]$
9. do if $v \in Q$ and $(d[u] + w(u,v) < d[v])$
10. then $\pi[v] \leftarrow u$
11. $d[v] \leftarrow d[u] + w(u,v)$ // Relax (u,v)
12. decrease_Key($Q, v, d[v]$)



Decrease-Key(Q, v, d[v])



$d[e] = \infty$
 $d[d] + w(d, e) = 2 + 5 = 7$ // $a \rightarrow d \rightarrow e$



Decrease-Key

- Binary heap:
 - $O(V)$ for search + $O(1)$ update value (once found) + $O(\log V)$ recover heap property (sift-up)
- Indexed heap (heap + pos[] map):
 - $O(1)$ search + $O(1)$ update value (once found) + $O(\log V)$ recover heap property + $O(1)$ update pos
- Other choices:
 - Fibonacci Heap:
 - $O(1)$ amortized decrease-key operations (theoretically optimal)
 - $O(\log V)$ amortized
 - Amortized time complexity represents the average cost of an operation over a long sequence of actions
 - Pairing Heap: Practical alternative to Fibonacci heap, decrease-key empirically close to $O(1)$
 - d-ary Heap:
 - increasing d makes the tree much shorter $\rightarrow$ decrease-key faster
 - however, slower extract-min
 - Lazy approach: instead of decrease-key, insert a new (v, newKey) , ignore the old entry

Time Complexity

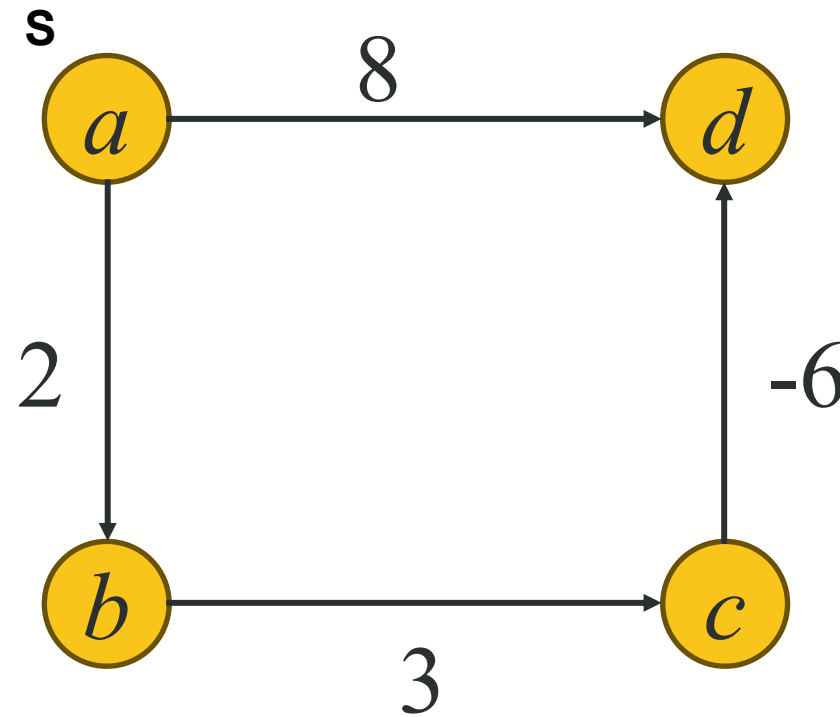
- Extracting the minimum-distance vertex: $O(\log V)$
- Edge relaxation update (decrease_key): $O(\log V)$

- Total cost:

- Processing V vertices $O(V \log V)$
 - Processing E edges $O(E \log V)$
 - Total $O((V+E) \log V)$
- Time complexity of Dijkstra's algorithm depends on the data structure used for the priority queue.
- $O(V^2)$ when a simple array is used instead of a priority Queue. (Extract-Min $O(V)$ performs V times)

```
6. while  $Q \neq \emptyset$ 
7.   do  $u \leftarrow \text{Extract-Min}(Q)$ 
       $S \leftarrow S \cup \{u\}$ 
8.   for each  $v \in \text{adj}[u]$ 
9.     do if  $v \in Q$  and  $(d[u] + w(u,v) < d[v])$ 
10.      then  $\pi[v] \leftarrow u$ 
11.       $d[v] \leftarrow d[u] + w(u,v)$  // Relax ( $u,v$ )
12.      decrease_Key( $Q, v, d[v]$ )
```


Negative Weights (1)



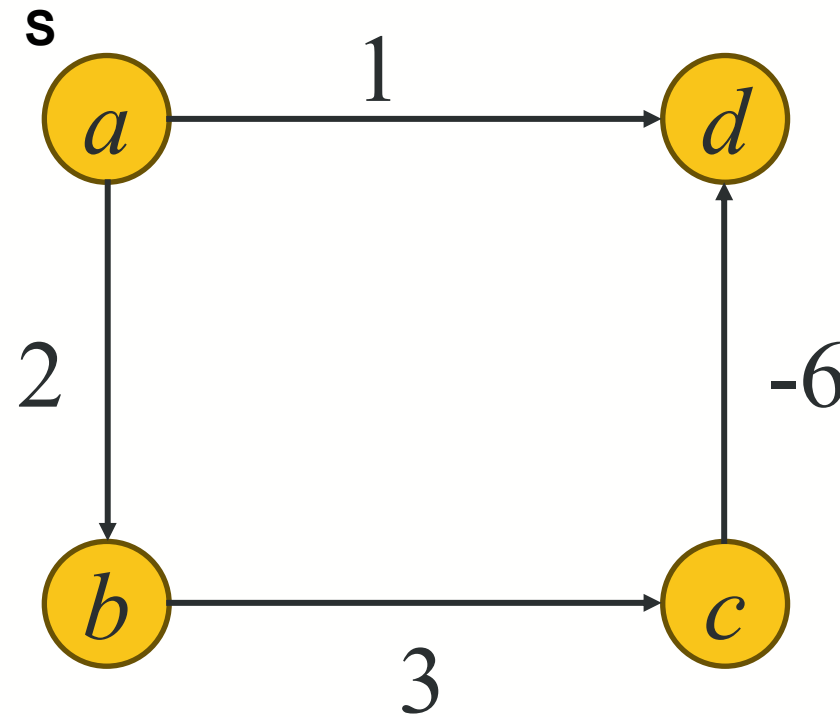
$S=\{a\}, V=\{d, b, c\}$
 $d[a]=0, d[b]=2, d[d]=8, d[c]=\infty$

$S=\{a, b\}, V=\{c, d\}$
 $d[a]=0, d[b]=2, d[d]=8, d[c]=5$

$S=\{a, b, c\}, V=\{d\}$
 $d[a]=0, d[b]=2, d[d]=-1, d[c]=5$

$S=\{a, b, c, d\}, V=\{\}$
 $d[a]=0, d[b]=2, d[d]=-1, d[c]=5$

Negative Weights (2)



$S=\{a\}, V=\{d, b, c\}$
 $d[a]=0, d[b]=2, d[d]=1, d[c]=\infty$

$S=\{a, d\}, V=\{b, c\}$
 $d[a]=0, d[b]=2, d[d]=1, d[c]=\infty$

$S=\{a, d, b\}, V=\{c\}$
 $d[a]=0, d[b]=2, d[d]=1, d[c]=5$

$S=\{a, d, b, c\}, V=\{\}$
 $d[a]=0, d[b]=2, d[d]=1, d[c]=5$

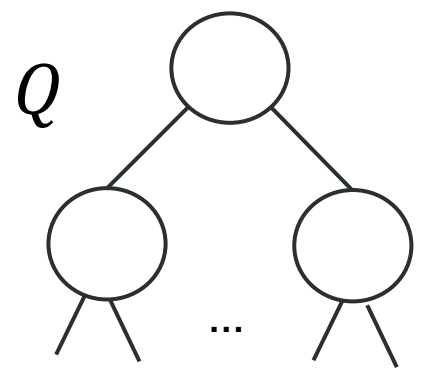
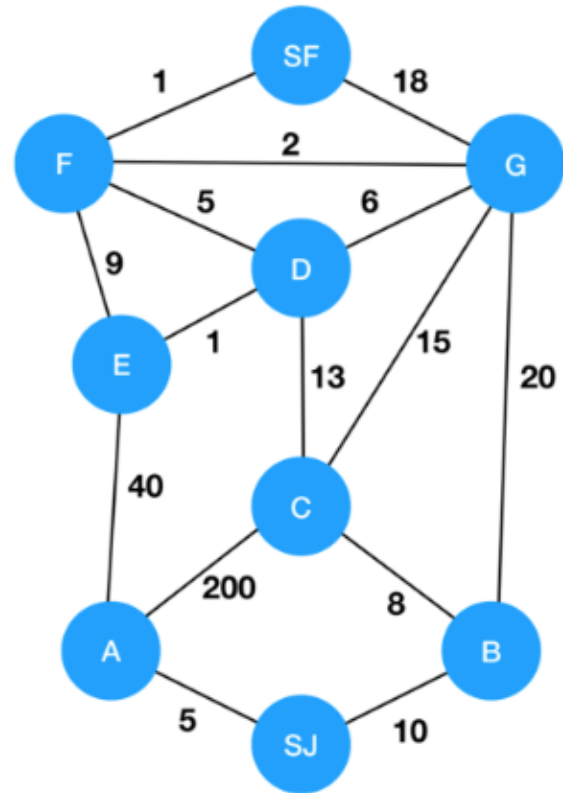
Dijkstra never revisits nodes after processing them, so it fails to correct distances if a negative edge provides a better path later.

Dijkstra's algorithm for undirected graphs

- Dijkstra's algorithm works for both directed and undirected graphs, as long as:
 - All edge weights are non-negative
 - The graph is connected
- In undirected graphs, store each undirected edge as two directed edges in your adjacency list
 - for edge (u,v) , include both (u, v) and (v, u)
- If all edge weights are equal (e.g., 1), use Breadth-first search (BFS) instead.
 - BFS gives shortest paths in unweighted graphs in $O(V+E)$ time
 - Dijkstra runs in $O((V+E)\log V)$ time using a priority queue

Try it yourself

- Work through Dijkstra's algorithm step by step. Fill in the table with the d and π , and show the contents of the heap Q at each step.



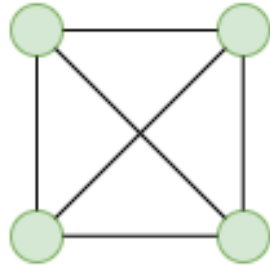
	d	π
SF		
F		
G		
D		
E		
C		
A		
B		
SJ		

Minimum Spanning Trees

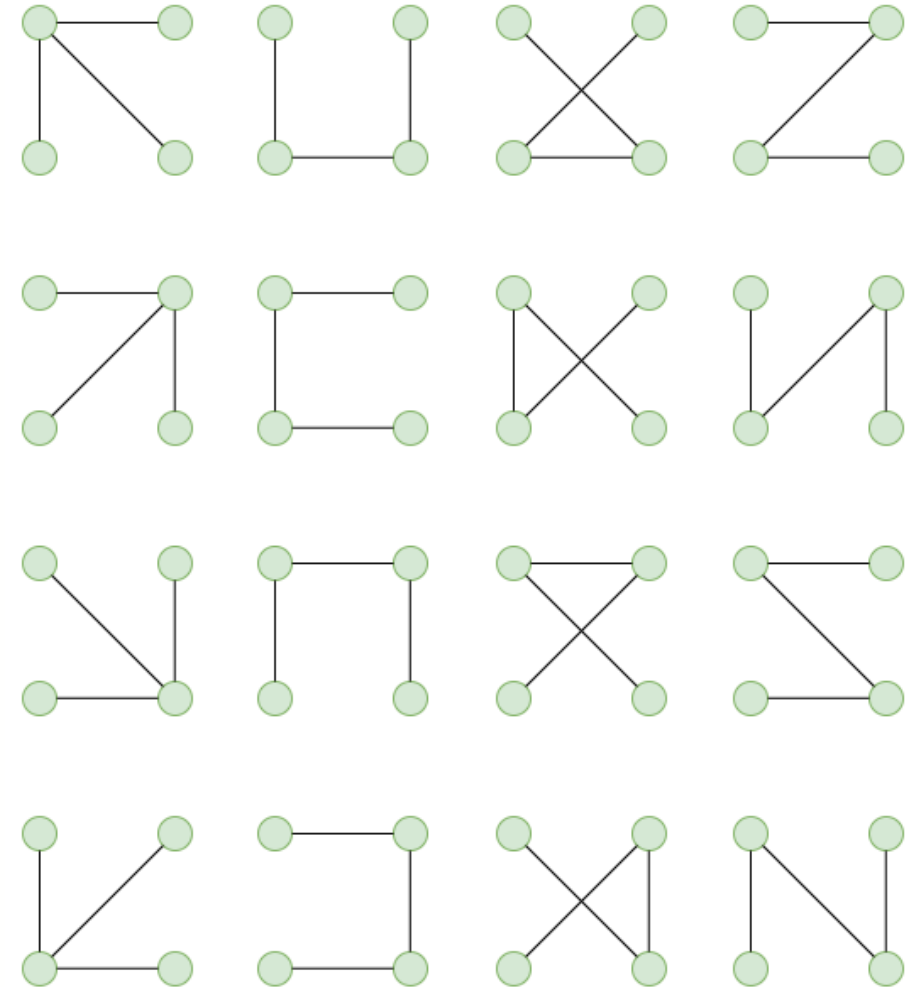
Slide Credit: These slides are adapted from materials by Prof. Manocha, with modifications and additions by Sujeong Kim.

Spanning Trees

Graph =



- A spanning tree connects all the nodes in a graph with the minimum number of edges required, ensuring there are no cycles.

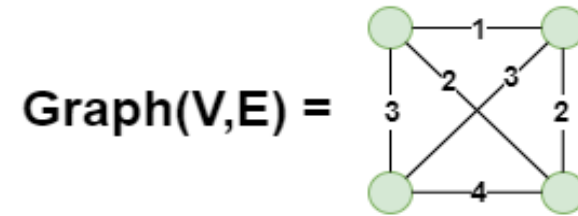


No. Edges = $|V|-1$

Facts about (Free) Trees

- A tree with n vertices has exactly $n-1$ edges
($|E| = |V| - 1$)
 - A tree is a minimally connected graph
- There exists a unique path between any two vertices of a tree
- Adding any edge to a tree creates a unique cycle; breaking any edge on this cycle restores a tree
- For details see CLRS Appendix B.5.1

Minimum Spanning Trees



Minimum Edge Weight: The sum of the weights of the edges in the tree is the smallest possible compared to any other spanning tree in the graph.

Motivation: Minimum Spanning Trees

- To minimize the length of a connecting network: connect all points (vertices) in a network with the least total length (or cost) of edges. Cycles involve extra edges that increase the total length without improving connectivity.
- The resulting connection graph is connected, undirected, and acyclic, i.e., a *free tree* (sometimes called simply a *tree*).
- This is the *minimum spanning tree* or *MST* problem.

Formal Definition of MST

- Given a connected, undirected, graph $G = (V, E)$, a *spanning tree* is an *acyclic* subset of edges $T \subseteq E$ that connects all the vertices together.
- Assuming G is weighted, we define the *cost* of a spanning tree T to be the sum of edge weights in the spanning tree

$$w(T) = \sum_{(u,v) \in T} w(u, v)$$

- A *minimum spanning tree (MST)* is a spanning tree of minimum weight.

Figure1 : Examples of MST

- Not only do the edges sum to the same value, but the same set of edge weights appear in the two MSTs. NOTE: An MST may not be unique.

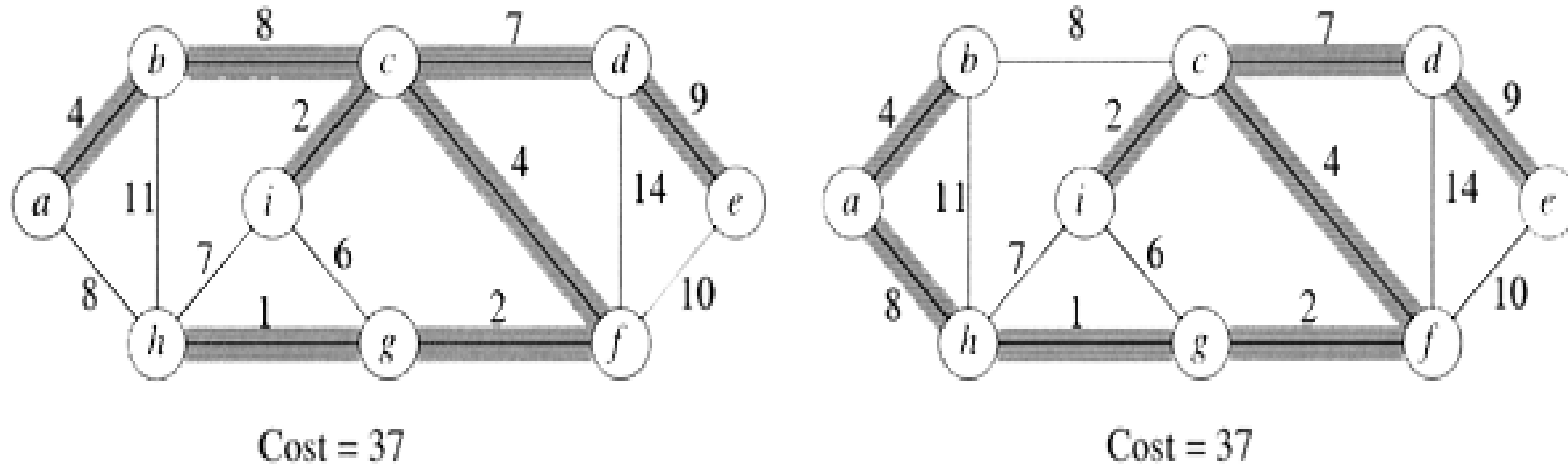


Figure 1: Minimum spanning tree.

Generic Approaches

- Two greedy algorithms for computing MSTs:
 - Kruskal's Algorithm (similar to connected component)
 - Prim's Algorithm (similar to Dijkstra's Algorithm)

Intuition Behind Greedy MST

- Subset of Edges A: This is a group of edges we are collecting to eventually form the MST. Initially this subset is empty.
- We will add edges to A one at a time.
- We say that a subset $A \subseteq E$ is **viable** if A does not violate the MST properties (connected, acyclic, minimal total weight) and can still be extended form a complete MST.
- We say that an edge $(u, v) \in E - A$ is **safe** if $A \cup \{(u, v)\}$ is viable.
- Basically, the choice (u, v) is a safe choice to add so that A can still be extended to form an MST.
- A generic greedy algorithm operates by repeatedly adding any **safe edge** to the current spanning tree.

Generic-MST (G, w)

1. $A \leftarrow \emptyset$ *// A trivially satisfies invariant*

// lines 2-4 maintain the invariant

2. while A does not form a spanning tree

3. do find an edge (u,v) that is safe for A

4. $A \leftarrow A \cup \{(u,v)\}$

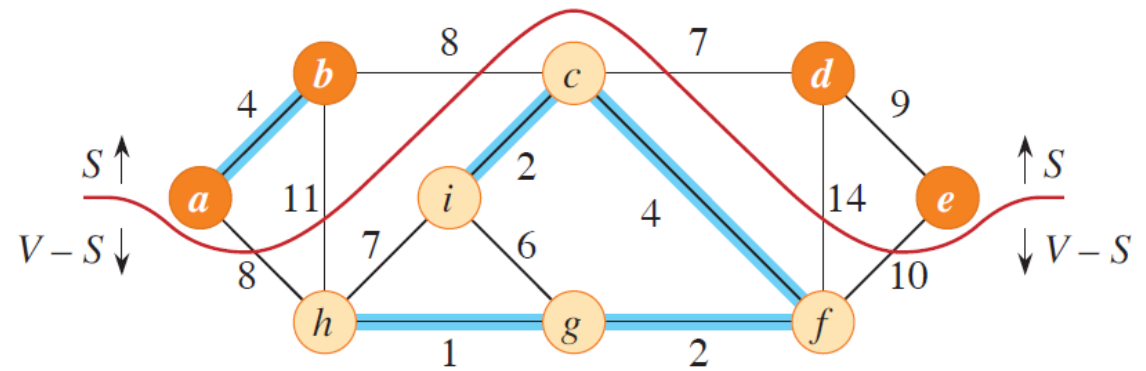
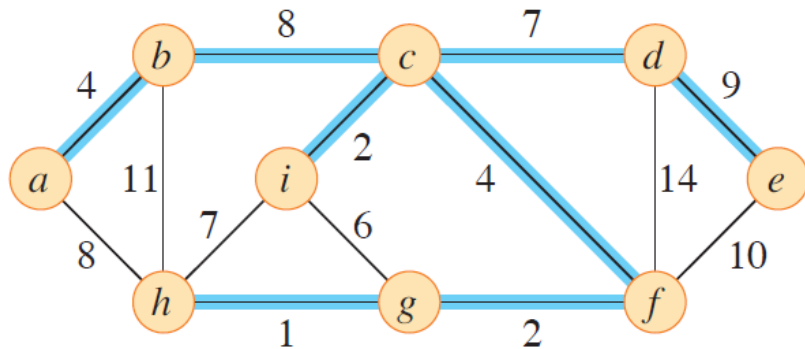
5. return A *// A is now a MST*

Ensuring Minimum Weight

- Choice of Safe Edge:
 - Kruskal's Algorithm
 - Prim's Algorithm

Definitions

- A *cut* $(S, V-S)$ is just a partition of the vertices into 2 disjoint subsets.
- An edge (u, v) *crosses* the cut if one endpoint is in S and the other is in $V-S$.
- Given a subset of edges A , we say that a cut *respects* A if no edge in A crosses the cut.



Definitions

- An edge of E is called a *light edge* crossing a cut if:
 - It crosses the cut (i.e., connects a vertex in S to a vertex in $V-S$).
 - It has the minimum weight among all edges that cross the cut.
 - There may be multiple light edges if there are edges with the same minimum weights.

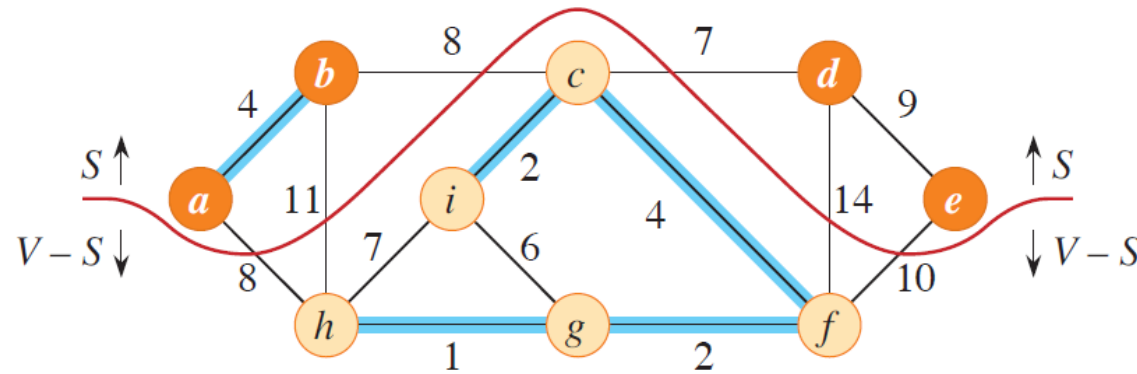


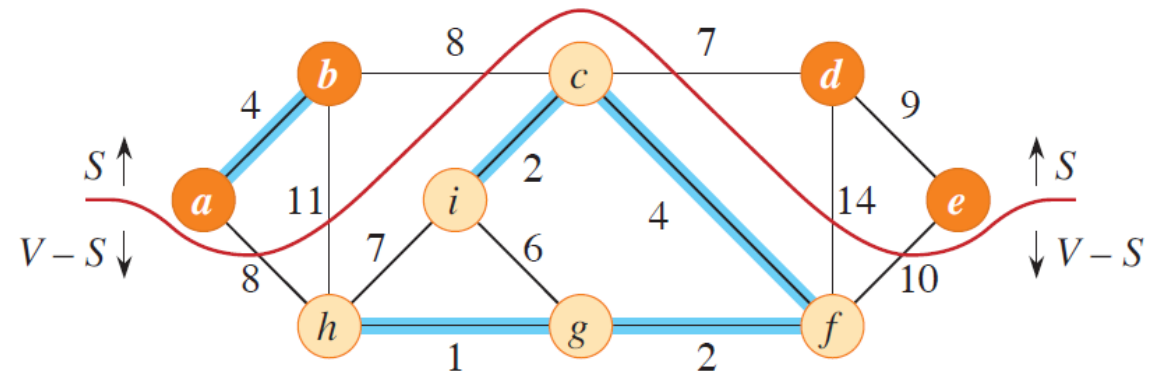
Figure 21.2 A cut $(S, V - S)$ of the graph from Figure 21.1. Orange vertices belong to the set S , and tan vertices belong to $V - S$. The edges crossing the cut are those connecting tan vertices with orange vertices. The edge (d, c) is the unique light edge crossing the cut. Blue edges form a subset A of the edges. The cut $(S, V - S)$ respects A , since no edge of A crosses the cut.

When is an Edge Safe?

- If we have computed a partial MST, and we wish to know which edges can be added that do NOT induce a cycle in the current MST, any edge that crosses a respecting cut is a possible candidate.
- Intuition says that since all edges crossing a respecting cut do not induce a cycle, then the lightest edge crossing a cut is a natural choice.

MST Lemma

- Let $G = (V, E)$ be a connected, undirected graph with real-value weights on the edges.
- Let A be a viable subset of E (i.e. a subset of some MST)
- Let $(S, V-S)$ be any cut that respects A
- Let (u, v) be a light edge crossing this cut.
- Then, the edge is safe for A .



Proof of MST Lemma

- Must show that $A \cup \{(u,v)\}$ is a subset of some MST
- Method:
 1. Find arbitrary MST T containing A
 2. Use a cut-and-paste technique to find another MST T' that contains $A \cup \{(u,v)\}$
- This cut-and-paste idea is an important proof technique

Figure

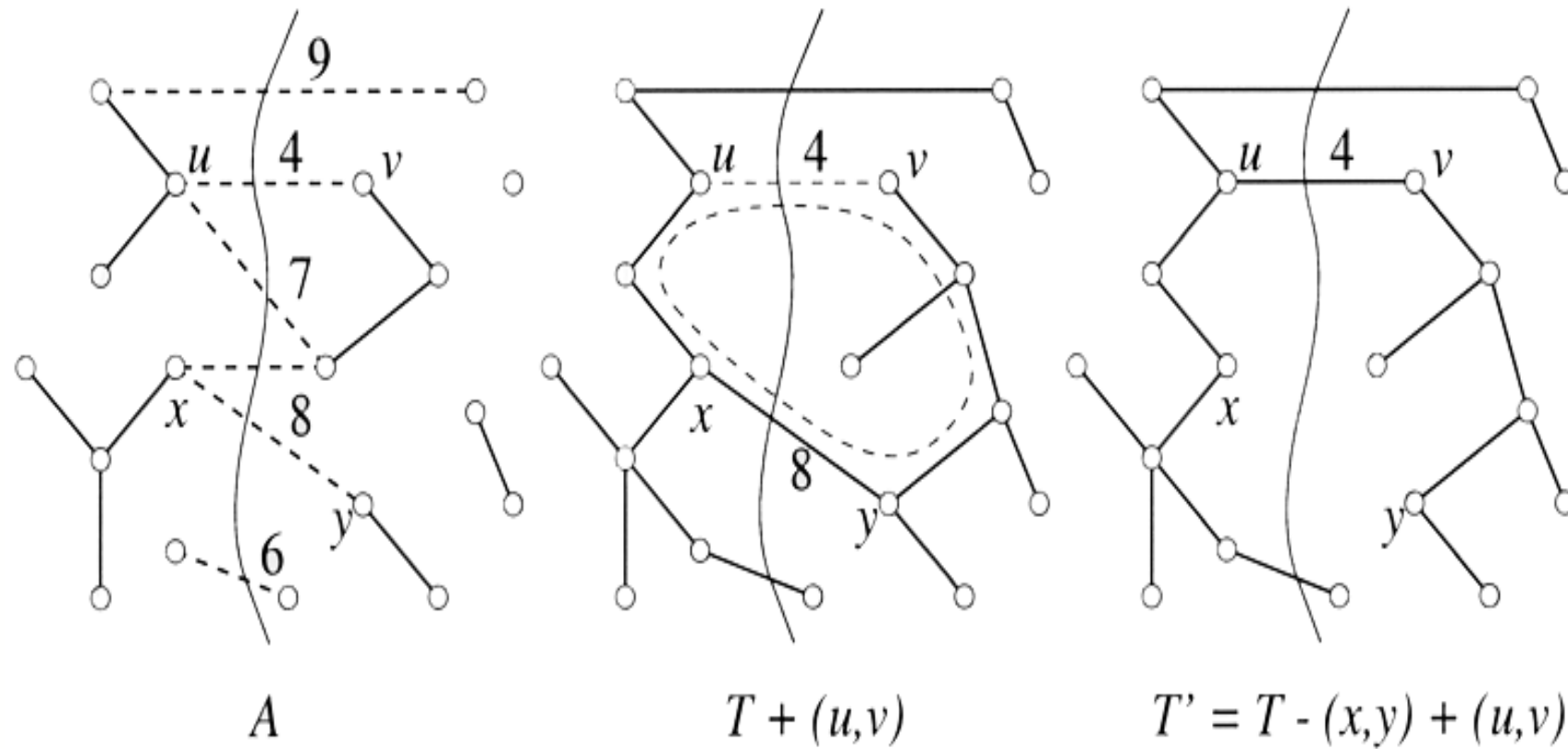


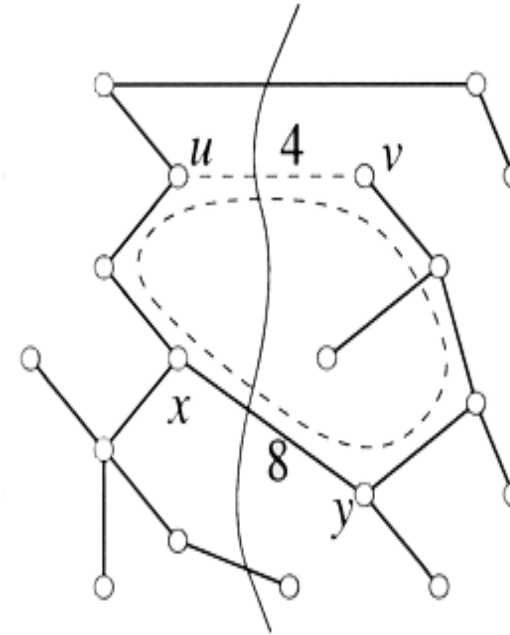
Figure 3: MST Lemma

Step 1

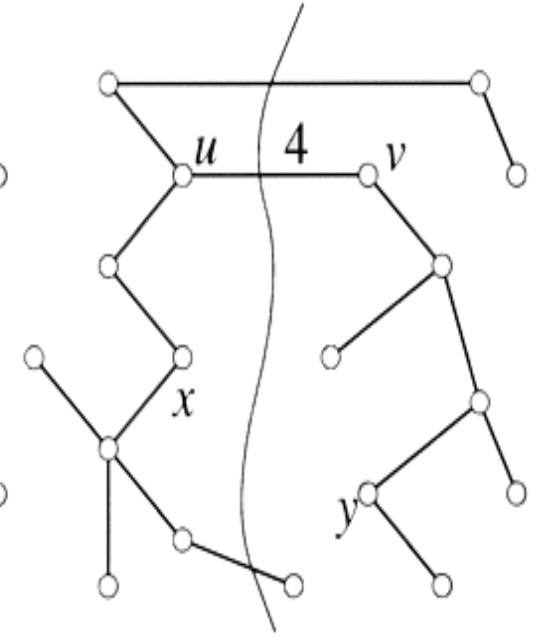
- Let T be any MST for G containing A .
 - We know such a tree exists because A is viable.
- If (u, v) is in T then we are done.

Constructing T'

- If (u, v) is not in T , then add it to T , thus creating a cycle.
- Since u and v are on opposite sides of the cut, and since any cycle must cross the cut an even number of times, there must be at least one other edge (x, y) in T that crosses the cut.



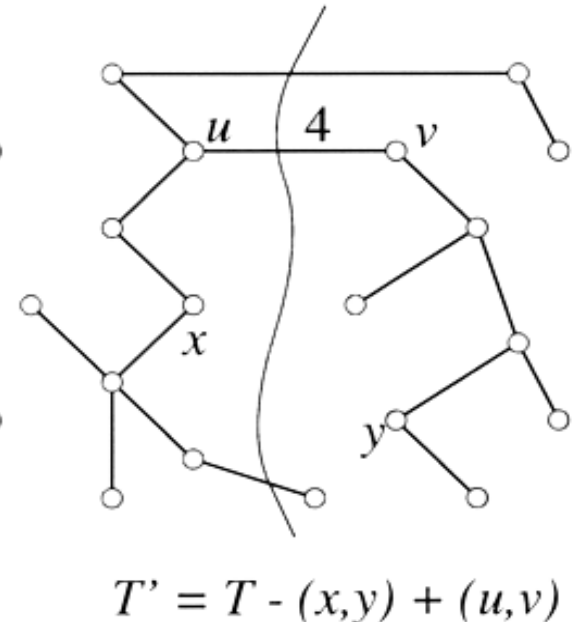
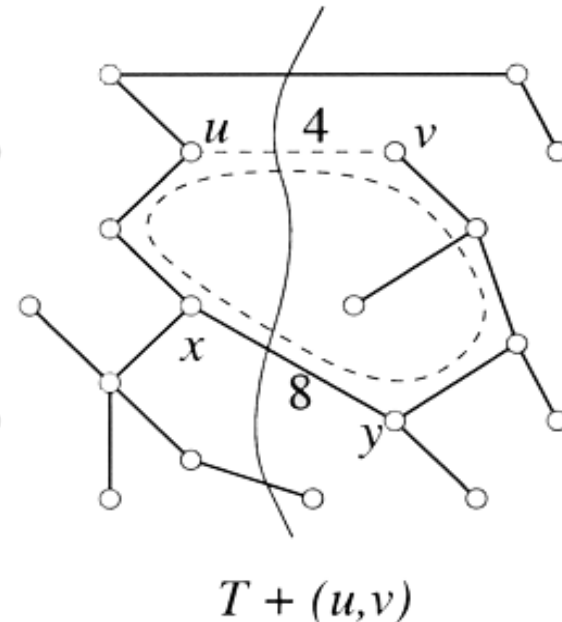
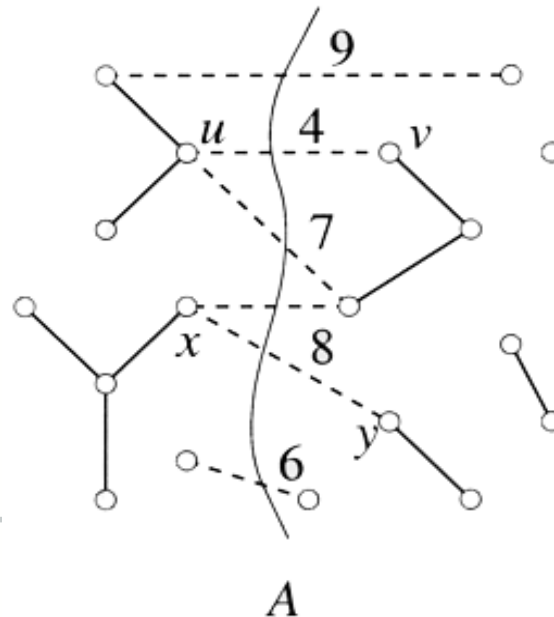
$T + (u, v)$



$T' = T - (x, y) + (u, v)$

Constructing T'

- The edge (x, y) is not in A (because the cut respects A). By removing (x, y) we restore a spanning tree, T' .
- Now must show
 - T' is a *minimum* spanning tree
 - $A \cup \{(u, v)\}$ is a subset of T'



Conclusion of Proof

- T' is a new spanning tree formed by modifying T .
- T' is an MST: We have

$$w(T') = w(T) - w(x, y) + w(u, v)$$

Since (u, v) is a light edge crossing the cut, we have $w(u, v) \leq w(x, y)$.

Thus $w(T') \leq w(T)$. So T' is also a minimum spanning tree.

- $A \cup \{(u, v)\} \subseteq T'$: Remember that (x, y) is not in A . Thus $A \subseteq T - \{(x, y)\}$, and thus

$$A \cup \{(u, v)\} \subseteq T - \{(x, y)\} \cup \{(u, v)\} = T'$$

MST Lemma: Reprise

- Let $G = (V, E)$ be a connected, undirected graph with real-value weights on the edges. Let A be a viable subset of E (i.e. a subset of some MST), let $(S, V-S)$ be any cut that respects A , and let (u, v) be a light edge crossing this cut. Then, the edge is safe for A .
- *Point of Lemma:* Greedy strategy works!

Basics of Kruskal's Algorithm

- Attempts to add edges to A in increasing order of weight (lightest edge first)
- If the next edge does not induce a cycle among the current set of edges, then it is added to A .
- If it does, then this edge is passed over, and we consider the next edge in order.
- As this algorithm runs, the edges of A will induce a forest on the vertices and the trees of this forest are merged together until we have a single tree containing all vertices.

Kruskal MST Visualization

Kruskal Minimum Cost Spanning Treeh

Run Kruskal

New Graph

- ☒ Small Graph ☒ Logical Representation
☐ Large Graph ☐ Adjacency List Representation
☐ Adjacency Matrix Representation

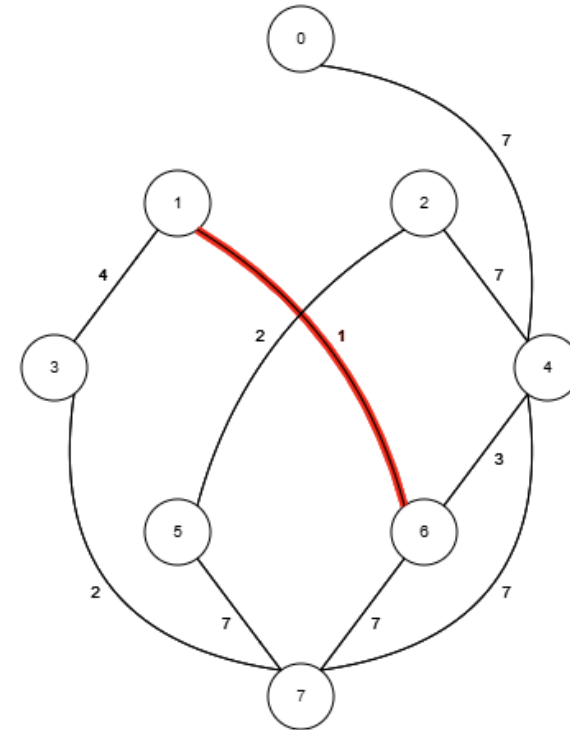
find(1) = 1 find(6) =

Disjoint Set

0	-1
1	-1
2	-1
3	-1
4	-1
5	-1
6	-1
7	-1

1	6
2	5
3	7
4	6
1	3
0	4
2	4
4	7
5	7
6	7

Animation Paused



Detecting a Cycle

- We can perform a DFS on subgraph induced by the edges of A , but this takes too much time.
- Use “disjoint set UNION-FIND” data structure. This data structure supports 3 operations:
 - Create-Set(u): create a set containing u .
 - Find-Set(u): Find the set that contains u .
 - Union(u, v): Merge the sets containing u and v .Each can be performed in $O(\lg n)$ time.
- The vertices of the graph will be elements to be stored in the sets; the sets will be vertices in each tree of A (stored as a simple list of edges).

MST-Kruskal(G, w)

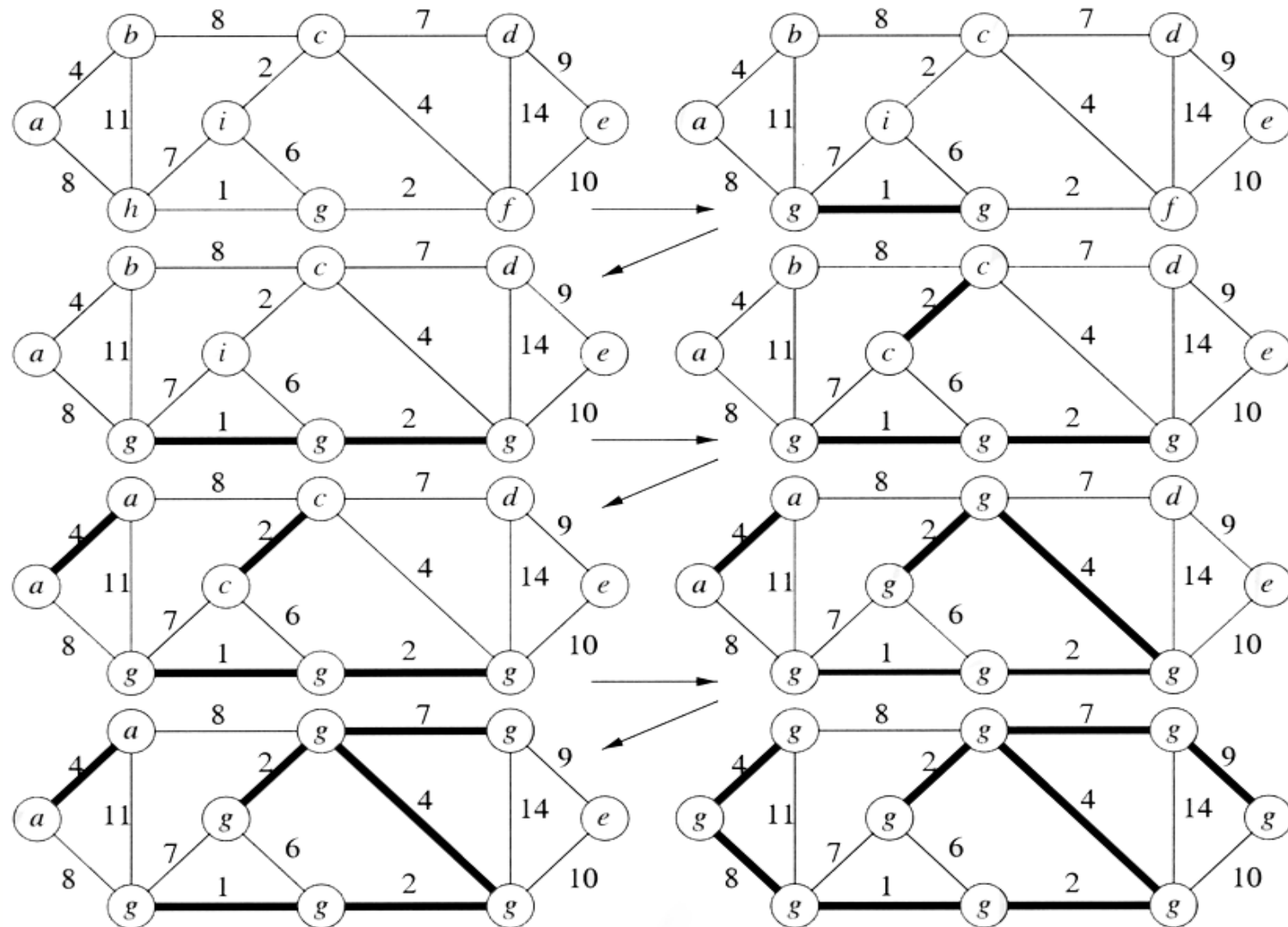
1. $A \leftarrow \emptyset$ // initially A is empty
2. for each vertex $v \in V[G]$ // line 2-3 takes $O(V)$ time
3. do Create-Set(v) // create set for each vertex
4. sort the edges of E by nondecreasing weight w
5. for each edge $(u,v) \in E$, in order by nondecreasing weight
6. do if Find-Set(u) $\neq$ Find-Set(v) // u & v on different trees
7. then $A \leftarrow A \cup \{(u,v)\}$
8. Union(u,v)
9. return A

Total running time is $O(E \lg E)$.

Analysis of Kruskal

- Lines 1-3 (initialization): $O(V)$
- Line 4 (sorting): $O(E \lg E)$
- Lines 6-8 (set-operation): $O(E \log E)$
- Total: $O(E \log E)$

Example: Kruskal's Algorithm



Kruskal's Algorithm

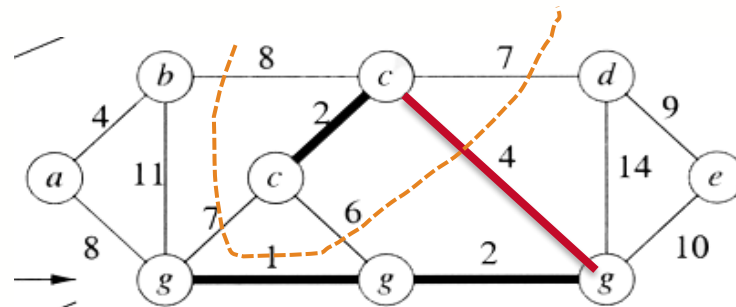
- Kruskal's algorithm sorts all the edges and then adds the smallest edge to the MST, provided it doesn't form a cycle. This continues until the MST includes all vertices.
- This algorithm is particularly efficient for sparse graphs where the number of edges E is much less than V^2 (the number of vertices squared).

Correctness

- Consider the edge (u, v) that the algorithm seeks to add next, and suppose that this edge does not induce a cycle in A . Let A' denote the tree of the forest A that contains vertex u . Consider the cut $(A', V - A')$. Every edge crossing the cut is not in A , and so this cut respects A , and (u, v) is the light edge across the cut (because any lighter edge would have been considered earlier by the algorithm). Thus, by the MST Lemma, (u, v) is safe.

Correctness of Kruskal

- Idea: Show that every edge added is a safe edge for A
- Assume (u, v) is next edge to be added to A .
 - Will not create a cycle
- Let A' denote the tree of the forest A that contains vertex u . Consider the cut $(A', V - A')$.
 - This cut respects A (why?)
 - and (u, v) is the light edge across the cut (why?)
- Thus, by the MST Lemma, (u, v) is safe.



Prim's Algorithm

- Start with an arbitrary vertex and include it in the MST.
- Initialize a priority queue to store edges with their weights.
- Repeat until all vertices are included in the MST:
 - Select the edge with the minimum weight that connects a vertex in the MST to a vertex outside the MST
 - Add this edge to the MST
 - Add all edges from this newly included vertex to the priority queue

Example: Prim's Algorithm

